

ESTACIÓN AMBIENTAL INTELIGENTE

Informe Solemne 1 - Programación de Microprocesadores

Carrera	Ingeniería Civil Informática
Asignatura	CINF1P09 Programación de Microprocesadores
Docente	Efraín Campusano
Plataforma	Arduino Uno
Fecha	Septiembre de 2026

Integrantes

Uriel Navarrete
Javier Farías
Javiera Morán
Guillermo Navarro
Vicente Abarca
Gianella Catalán

Resumen

Se diseñó e implementó una estación ambiental basada en Arduino Uno para adquirir de forma sincronizada temperatura, humedad relativa, presión atmosférica, iluminancia, respuesta analógica del MQ135 y eventos acústicos. El diseño y el programa integrado contemplan los cinco sensores asignados. Durante la validación experimental, DHT11, BH1750, MQ135 y el módulo de sonido entregaron datos utilizables, mientras que el BMP180 no produjo una respuesta funcional. En consecuencia, el software mantiene la variable `presion_hPa` y continúa ejecutándose cuando el BMP180 no está disponible, registrando NaN en dicha columna en lugar de generar valores artificiales. El conjunto consolidado disponible contiene 420 registros: 300 ciclos de adquisición integrada a aproximadamente 1 Hz y 120 registros complementarios del MQ135.

Contenido

1. Contextualización y objetivos
2. Arquitectura general del sistema
3. Fundamentos teóricos de los sensores
4. Diagramas y conexiones
5. Aplicación práctica: funcionamiento individual
6. Programa integrado
7. Desarrollo experimental y resultados
8. Adquisición y archivo CSV
9. Explicación del flujo del código integrado
10. Aporte individual
11. Conclusiones individuales
12. Bibliografía
13. Anexo

1. Contextualización y objetivos

1.1 Contextualización

El proyecto corresponde al desarrollo de una estación de monitoreo ambiental para un recinto interior. El sistema utiliza Arduino Uno como unidad central y combina cinco sensores de distinta naturaleza: DHT11 para temperatura y humedad relativa, BMP180 para presión atmosférica, BH1750 para iluminancia, MQ135 para observar la variación de una señal analógica asociada a la calidad del aire y un módulo de sonido digital para registrar eventos acústicos por umbral. La dificultad principal consiste en integrar señales digitales, analógicas e I2C en una adquisición temporal común y generar un registro CSV trazable.

1.2 Objetivo general

Diseñar, implementar, documentar y validar un sistema de adquisición de datos basado en Arduino Uno que contemple los cinco sensores asignados, registre las variables disponibles por puerto serial y almacene las mediciones reales en formato CSV, documentando de forma explícita cualquier limitación experimental.

1.3 Objetivos específicos

- Comprender el principio físico y electrónico de cada sensor y relacionarlo con la señal procesada por Arduino Uno.
- Validar cada sensor mediante un programa independiente antes de utilizar el programa integrado.
- Integrar sensores digitales, analógicos e I2C manteniendo una muestra conjunta aproximadamente cada 1 s.
- Distinguir lecturas crudas de variables físicas y evitar conversiones no justificadas, particularmente en el MQ135.
- Generar un CSV con datos reales y documentar el significado, unidad y origen de cada columna.

1.4 Alcance y limitaciones

La estación tiene finalidad académica y experimental. El MQ135 se analiza como lectura ADC cruda y no como concentración en ppm, ya que no se realizó una calibración que permita justificar dicha conversión. El módulo de sonido entrega un estado digital definido por un umbral ajustable y no constituye un sonómetro. El BMP180 se mantuvo contemplado en el programa integrado, pero el módulo utilizado no entregó una respuesta funcional durante las pruebas; por ello, la columna de presión del CSV contiene NaN.

2. Arquitectura general del sistema

Arduino Uno centraliza la adquisición y el envío serial. El DHT11 utiliza D2, el módulo de sonido utiliza D3 y el MQ135 utiliza la entrada analógica A0. El BMP180 y el BH1750 emplean comunicación I2C, pero en el montaje realizado se consideran conexiones o puertos físicos diferenciados; no se presentan como compartiendo un mismo puerto. La asignación físicoteóricos de los sensores

Para la clasificación activo/pasivo se adopta en este informe una convención electrónica: se considera activo al módulo que requiere alimentación externa y posee electrónica de acondicionamiento o conversión. Se explicita esta convención porque en instrumentación existen definiciones alternativas.

3. Fundamentos teóricos de los sensores

3.1 DHT11 - temperatura y humedad

El DHT11 es un sensor digital combinado de temperatura y humedad. El manual ASAIR V1.3 describe un elemento capacitivo para humedad, un termistor NTC para temperatura y electrónica interna que transmite una trama digital de 40 bits. La comunicación se realiza por una línea de datos y requiere respetar un intervalo de muestreo superior a 2 s para obtener lecturas estables.



Figura 1. Módulo DHT11 utilizado.

Fuente: fotografía propia del módulo utilizado en el montaje.

Parámetro	Descripción
Variable	Temperatura [°C] y humedad relativa [%RH].
Clasificación	Digital; activo bajo la convención adoptada.
Rango de temperatura	-20 a 60 °C (manual ASAIR V1.3).
Precisión de temperatura	±2 °C a 25 °C.
Rango de humedad	5 a 95 %RH.
Precisión de humedad	±5 %RH a 25 °C.
Tiempo de respuesta	<10 s temperatura; <6 s humedad (1/e, 63 %).
Alimentación	3,3 a 5,5 V DC para el sensor.

Parámetro	Descripción
Interfaz	Serial digital de una línea; DATA conectado a D2.
Muestreo	Periodo de medición >2 s según manual V1.3.
Biblioteca	DHT sensor library de Adafruit.

3.2 BMP180 - presión atmosférica

El BMP180 es un sensor barométrico digital MEMS piezoresistivo con conversión A/D, coeficientes individuales de calibración y comunicación I2C. La presión se obtiene aplicando el algoritmo de compensación del fabricante a las lecturas crudas de temperatura y presión. En el proyecto se preparó y ejecutó una prueba individual del sensor; sin embargo, el módulo físico no entregó una respuesta funcional y no se obtuvieron valores válidos de presión. Esta condición se documenta como una limitación experimental.

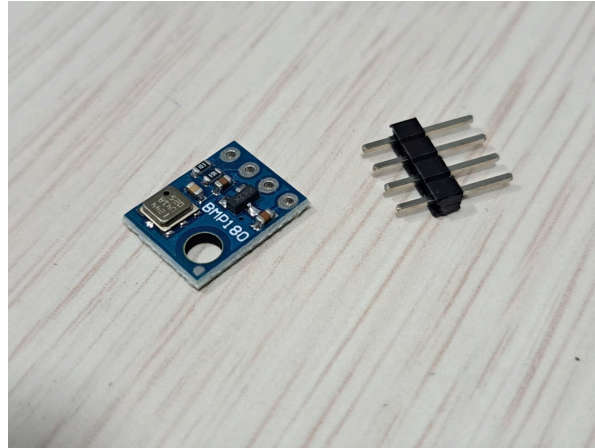


Figura 2. Módulo BMP180 fotografiado.

Fuente: fotografía propia del módulo utilizado en el montaje.

Parámetro	Descripción
Variable	Presión atmosférica [Pa/hPa]; temperatura interna usada para compensación.
Clasificación	Digital I2C; activo bajo la convención adoptada.
Dirección I2C	0x77.
Chip ID	Registro 0xD0; valor esperado 0x55 y lectura de un byte.
Rango de presión	300 a 1100 hPa.
Resolución de salida	0,01 hPa; no equivale a precisión.
Exactitud relativa	±0,12 hPa en condiciones específicas indicadas por Bosch.
Exactitud absoluta	El datasheet especifica límites dependientes de presión y temperatura; no se resume como ±0,01 hPa.
Ruido	Aproximadamente 0,02 hPa en modo de ultra alta resolución.
Conversión OSS=3	17 ms típicos; 25,5 ms máximos.
Alimentación del IC	VDD 1,8-3,6 V; VDDIO 1,62-3,6 V. El breakout debe distinguirse del IC.
Resultado experimental	Sin lectura válida; presion_hPa queda como NaN en el CSV.

3.3 BH1750 / GY-30 - iluminancia

El BH1750FVI es un sensor digital de luz ambiental con interfaz I2C. El programa utiliza comunicación directa mediante Wire, detecta las direcciones 0x23 y 0x5C y configura el modo continuo de alta resolución con el comando 0x10. La conversión utilizada es $\text{lux} = \text{valor_crudo} / 1,2$, siguiendo la nota técnica del dispositivo.

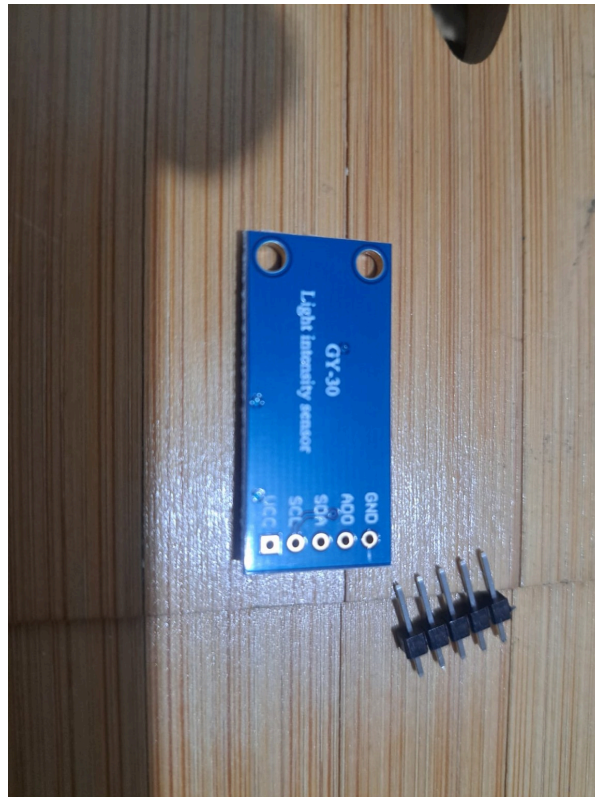


Figura 3. Módulo GY-30 con sensor BH1750.

Fuente: fotografía propia del módulo utilizado en el montaje.

Parámetro	Descripción
Variable	Iluminancia [lux].
Clasificación	Digital I2C; activo bajo la convención adoptada.
Direcciones	0x23 con ADDR bajo / 0x5C con ADDR alto.
Resolución usada	1 lx en modo H-Resolution.
Tiempo de medición	120 ms típico; 180 ms máximo en alta resolución.
Salida	Dos bytes (16 bits) por I2C.
Conversión	$\text{lux} = \text{valor_crudo} / 1,2$ en la configuración utilizada.
Interfaz Arduino	I2C; conexión física según el montaje y el diagrama integrado definitivo.
Biblioteca	Wire; no se usa una biblioteca BH1750 externa.

3.4 MQ135 - respuesta analógica de gases

El MQ135 utiliza un material semiconductor SnO₂ cuya conductividad cambia ante distintos gases y vapores. El módulo empleado entrega una salida analógica AO y una salida digital DO; en el proyecto se utiliza AO conectada a A0. La lectura de `analogRead()` se conserva como valor ADC crudo. El manual Winsen describe sensibilidad frente a amoníaco, sulfuros, vapores de la serie del benceno, hidrógeno y humo, y especifica un calentador de 5 V y un precalentamiento inicial superior a 48 h bajo condiciones de referencia.



Figura 4. Módulo MQ135 utilizado.

Fuente: fotografía propia del módulo utilizado en el montaje.

Parámetro	Descripción
Variable usada	mq135 raw: lectura ADC cruda.
Clasificación	Sensor semiconductor con salida analógica de módulo.
Material sensible	SnO ₂ .
Calentador	5,0 V; consumo indicado por Winsen ≤ 950 mW.
Pre calentamiento inicial	>48 h en condiciones de referencia del fabricante.
Interfaz del proyecto	AO -> A0. DO no participa en la adquisición.
ADC Arduino Uno	10 bits: 0 a 1023.
Conversión auxiliar	$V = \text{raw} \times 5 / 1023$.
Restricción	No se reportan ppm sin calibración demostrada.

3.5 Módulo de sonido digital

El módulo de sonido utilizado incorpora un micrófono electret, una etapa de acondicionamiento y un comparador LM393 con umbral ajustable mediante potenciómetro. La salida digital cambia de estado cuando la señal supera el umbral configurado. El programa registra el estado digital y, además, cuenta los cambios de estado por segundo como variable derivada.



Figura 5. Módulo de sonido con micrófono electret y comparador.
Fuente: fotografía propia del módulo utilizado en el montaje.

Parámetro	Descripción
Variable usada	Estado digital 0/1 y flancos por segundo.
Clasificación	Módulo digital por umbral; activo.
Elemento de detección	Micrófono electret.
Comparador	LM393, salida de comparador con umbral ajustable.
Interfaz	OUT -> D3.
Unidad	El estado 0/1 y el conteo de flancos no son una unidad física de nivel sonoro.
Limitación	La polaridad exacta 0/1 depende del módulo y del ajuste; se registra el estado observado sin convertirlo a dB.

4. Diagramas y conexiones

4.1 DHT11

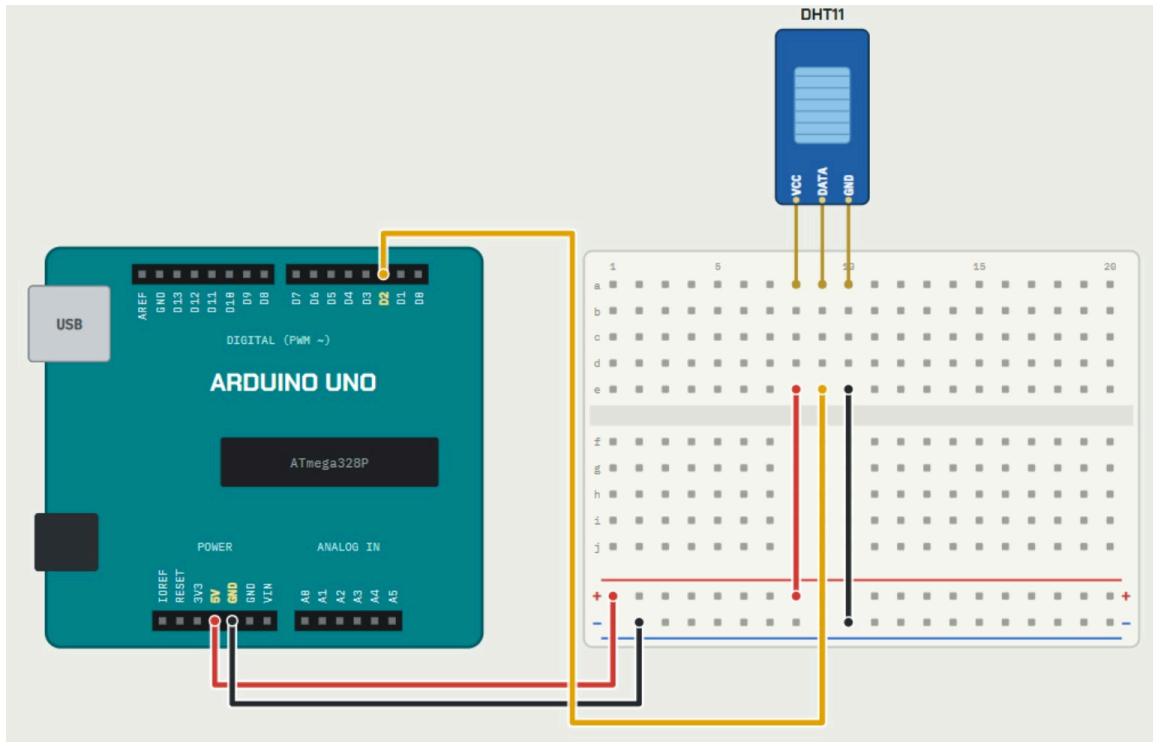


Figura 6. Conexión individual DHT11.

Fuente: elaboración propia.

4.2 BMP180

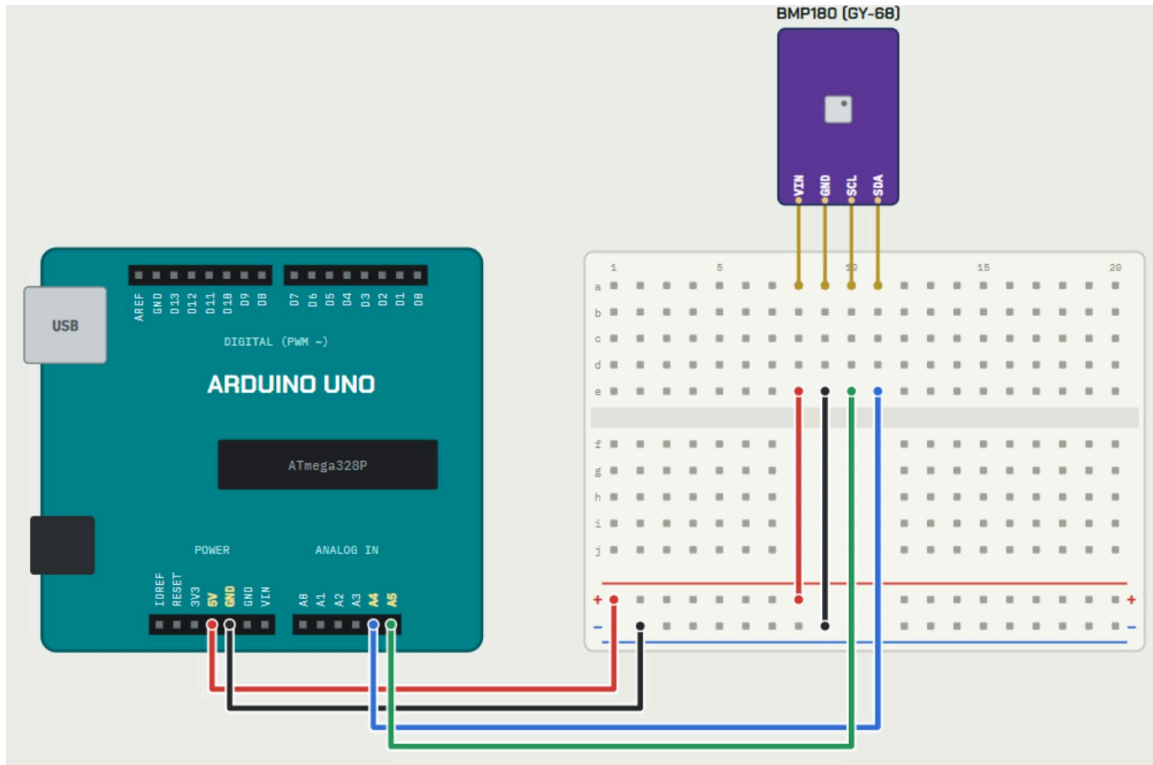


Figura 7. Conexión individual BMP180.

Fuente: elaboración propia.

4.3 BH1750

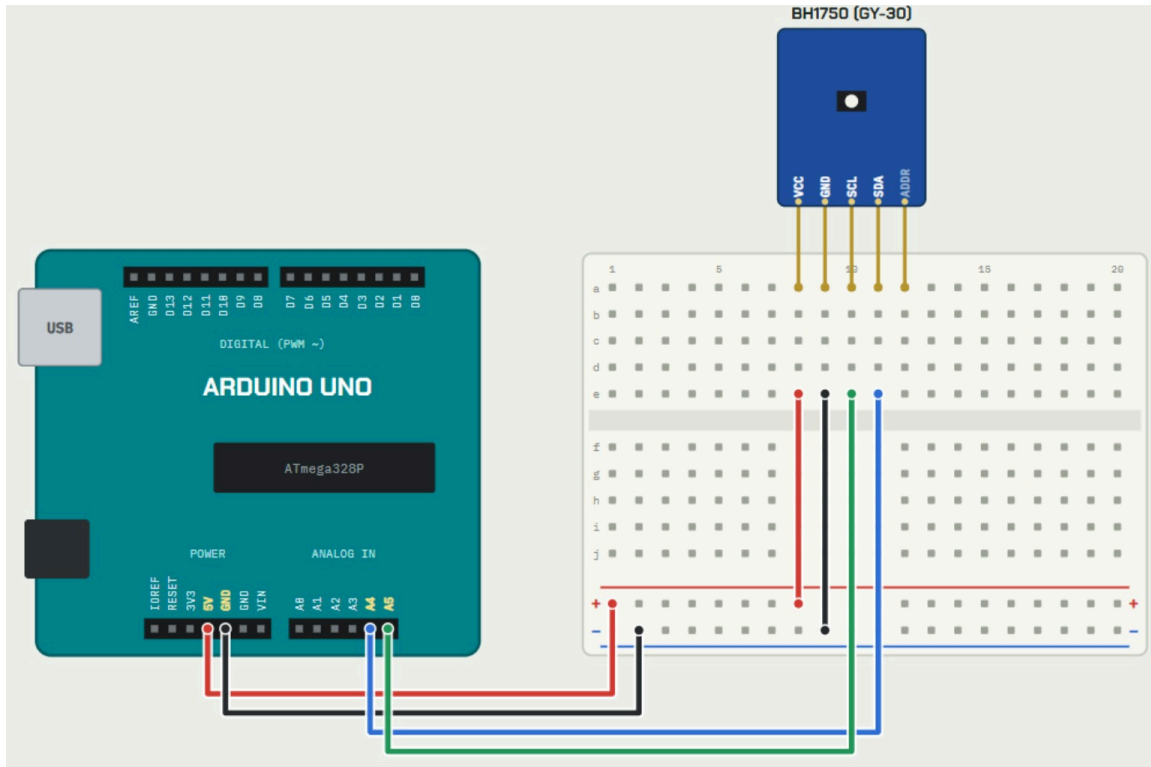


Figura 8. Conexión individual BH1750.

Fuente: elaboración propia.

4.4 MQ135

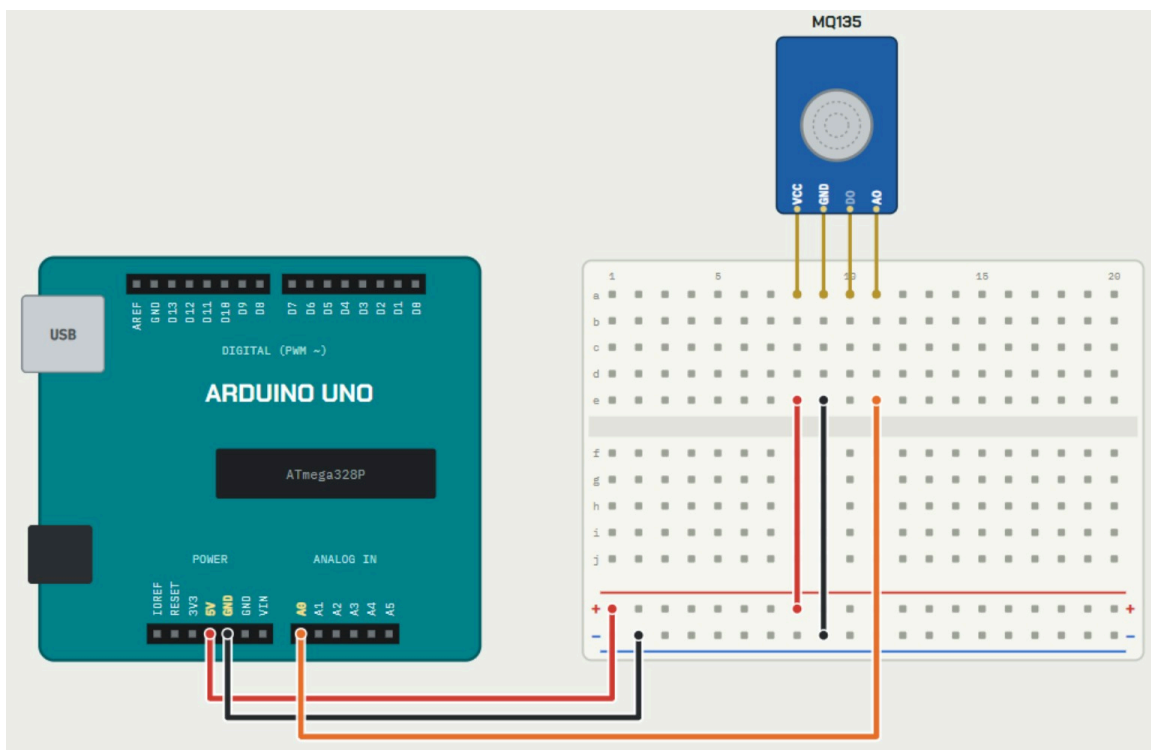


Figura 9. Conexión individual MQ135.

Fuente: elaboración propia.

4.5 Módulo de sonido

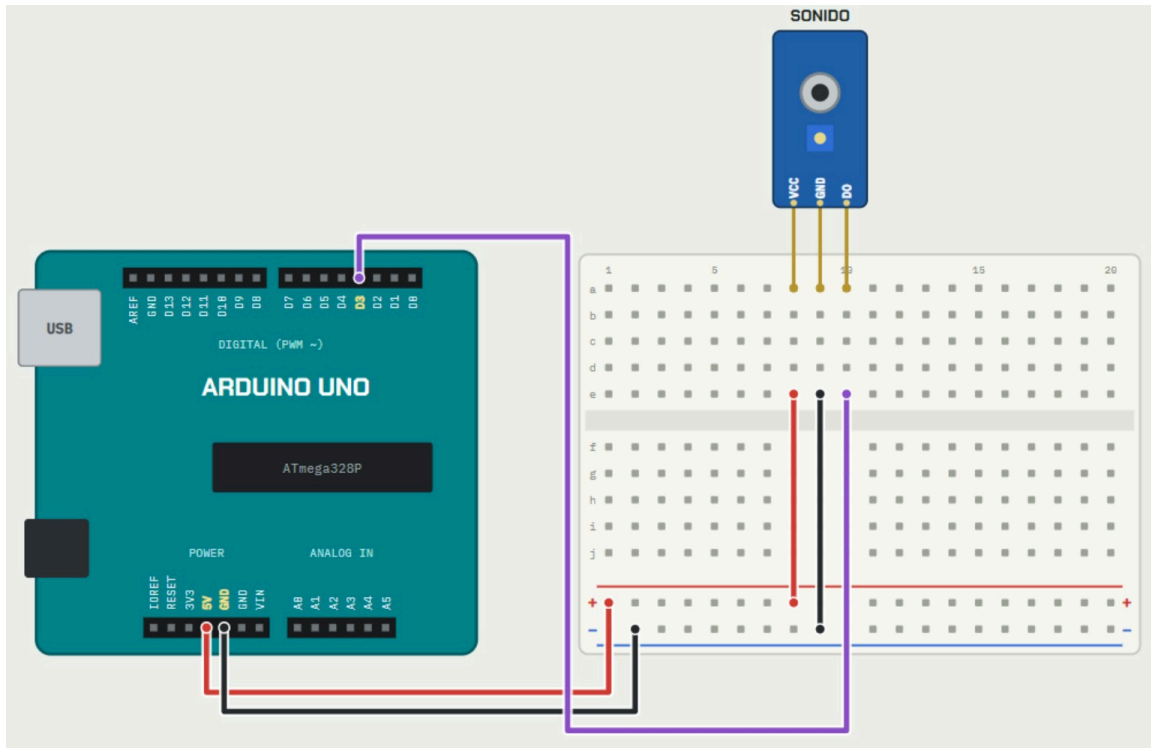


Figura 10. Conexión individual del módulo de sonido.

Fuente: elaboración propia.

4.6 Diagrama integrado

Se excluye el BMP180 debido a las fallas físicas mencionadas en el documento.

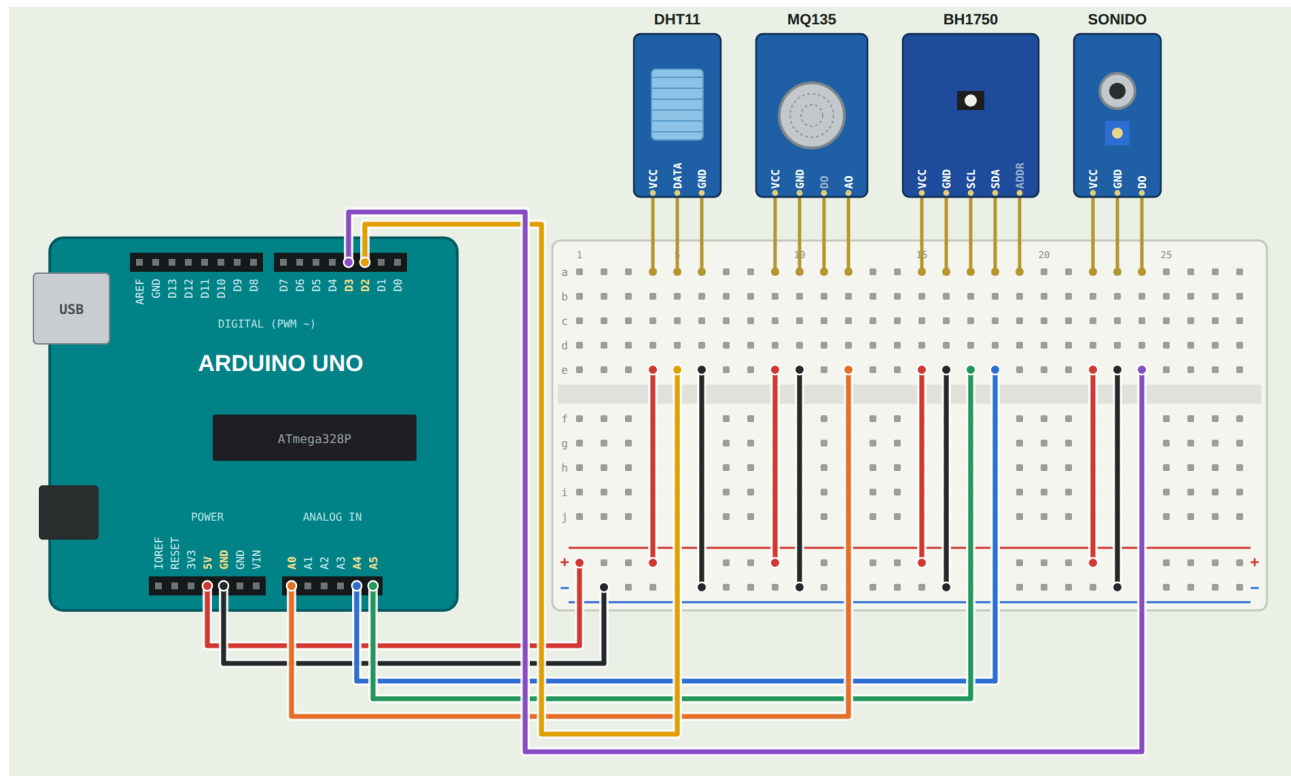


Figura 11. Conexión Diagrama integrado.

Fuente: elaboración propia.

5. Aplicación práctica: funcionamiento individual

Cada sensor fue abordado mediante un programa independiente antes de la integración. En el cuerpo del informe se muestran fragmentos representativos de la lógica. Los programas completos se entregan como archivos .ino independientes.

5.1 DHT11

La prueba inicializa el sensor en D2 con la biblioteca DHT y consulta temperatura y humedad cada 2 s. Las lecturas NaN se tratan como inválidas y no se presentan como mediciones reales. En la serie principal se obtuvieron 299 pares válidos de temperatura y humedad.

```

1  #define TIPO_DHT DHT11
2  DHT dht(PIN_DHT, TIPO_DHT);
3
4  void setup() {
5      Serial.begin(115200);
6      dht.begin();
7      Serial.println(F("tiempo_ms,temperatura_C,humedad_pct"));
8  }
9
10 void loop() {
11     static uint32_t t_ultima = 0;
12     if (millis() - t_ultima < 2000) return;
13     t_ultima = millis();
14
15     float humedad = dht.readHumidity();
16     float temperatura = dht.readTemperature();
17     if (isnan(humedad) || isnan(temperatura)) {
18         Serial.println(F("# DHT11: lectura invalida"));
19         return;
20     }
21
22     Serial.print(millis()); Serial.print(',');
23     Serial.print(temperatura, 1); Serial.print(',');
24     Serial.println(humedad, 1);
25 }

```

Figura 12. Fragmento de la prueba individual DHT11.

Fuente: elaboración propia, a partir del código fuente entregado (Zapalloman, 2026).

5.2 BMP180

La prueba individual del BMP180 utiliza comunicación I2C para intentar identificar el dispositivo y obtener la presión compensada. Durante las pruebas realizadas, el módulo no entregó valores funcionales. Esta ausencia de datos no afecta la ejecución del programa integrado ni las mediciones de los demás sensores; simplemente la variable de presión permanece sin un valor numérico válido.

```

// Fragmento del programa individual BMP180 utilizado en el proyecto
void setup() {
    Serial.begin(115200);
    Wire.begin();

    Wire.beginTransmission(DIR_BMP180);
    if (Wire.endTransmission() != 0) {
        Serial.println(F("# BMP180 no detectado. Revisar conexion y alimentacion."));
        while (true) { delay(1000); }
    }

    uint16_t id = leer16(0xD0);
    if (id != 0x55) {
        Serial.print(F("# BMP180: chip ID inesperado 0x"));
        Serial.println(id, HEX);
        while (true) { delay(1000); }
    }

    Serial.println(F("tiempo_ms,presion_hPa,bmp_temp_C"));
}

```

Figura 13. Fragmento de la prueba individual BMP180.

Fuente: elaboración propia, a partir del código fuente entregado (Zapalloman, 2026).

5.3 BH1750

El programa prueba las direcciones 0x23 y 0x5C, configura el modo continuo de alta resolución y lee dos bytes cada segundo. La iluminancia se obtiene mediante crudo/1,2. El sensor respondió correctamente a cambios de iluminación.

```

1  #include <Wire.h>
2  uint8_t dirBH = 0x23;
3
4  bool escribirBH(uint8_t comando) {
5      Wire.beginTransaction(dirBH);
6      Wire.write(comando);
7      return Wire.endTransmission() == 0;
8  }
9
10 void setup() {
11     Serial.begin(115200); Wire.begin();
12     bool encontrado = false;
13     uint8_t candidatas[2] = {0x23, 0x5C};
14     for (uint8_t i=0; i<2; i++) {
15         dirBH = candidatas[i];
16         if (escribirBH(0x01)) { encontrado = true; break; }
17     }
18     if (!encontrado) {
19         Serial.println(F("# BH1750 no detectado"));
20         while (true) delay(1000);
21     }
22     escribirBH(0x10); // continuo, alta resolucion
23     delay(180);
24     Serial.println(F("tiempo_ms,lux"));
25 }
26
27 void loop() {
28     static uint32_t t_ultima = 0;
29     if (millis() - t_ultima < 1000) return;
30     t_ultima = millis();
31     Wire.requestFrom(dirBH, (uint8_t)2);
32     if (Wire.available() < 2) return;
33     uint16_t crudo = ((uint16_t)Wire.read() << 8) | Wire.read();
34     float lux = crudo / 1.2;
35     Serial.print(millis()); Serial.print(',');
36     Serial.println(lux, 1);

```

Figura 14. Fragmento de la prueba individual BH1750.

Fuente: elaboración propia, a partir del código fuente entregado (Zapalloman, 2026).

5.4 MQ135

La prueba utiliza analogRead(A0) para obtener el valor ADC crudo y calcula un voltaje equivalente sólo como referencia eléctrica. No se realiza una conversión a ppm. El conjunto experimental muestra una línea base baja y una variación mayor en el ensayo complementario.

```

1  * Solemne 1 - MQ135. A0 -> A0, VCC -> 5 V, GND -> GND.
2  * Se reporta ADC crudo; no se convierte a ppm sin calibración.
3  * Winsen indica >48 h de precalentamiento inicial en condiciones de referencia.
4  */
5  #define PIN_MQ135 A0
6  void setup() {
7    Serial.begin(115200);
8    pinMode(PIN_MQ135, INPUT);
9    Serial.println(F("tiempo_ms,mq135_raw,mq135_volt"));
10 }
11 void loop() {
12   static uint32_t t_ultima = 0;
13   if (millis() - t_ultima < 1000) return;
14   t_ultima = millis();
15   int raw = analogRead(PIN_MQ135);
16   float voltaje = raw * (5.0 / 1023.0);
17   Serial.print(millis()); Serial.print(',');
18   Serial.print(raw); Serial.print(',');
19   Serial.println(voltaje, 3);
20 }

```

Figura 15. Fragmento de la prueba individual MQ135.

Fuente: elaboración propia, a partir del código fuente entregado (Zapalloman, 2026).

5.5 Módulo de sonido

La salida digital se muestrea cada 2 ms para detectar eventos breves. Cada cambio de estado incrementa un contador de flancos; una vez por segundo se reportan el estado actual y el total de cambios del intervalo. El conteo es una variable derivada y no equivale a cantidad de aplausos ni a decibeles.

```

1  * Solemne 1 - Modulo de sonido digital.
2  * Microfono electret + comparador LM393 + potenciómetro de umbral.
3  * OUT -> D3, VCC -> 5 V, GND -> GND.
4  */
5  #define PIN_SONIDO 3
6  void setup() {
7    Serial.begin(115200);
8    pinMode(PIN_SONIDO, INPUT);
9    Serial.println(F("tiempo_ms,estado_sonido,flancos_ultimo_segundo"));
10 }
11 void loop() {
12   static uint8_t estado_previo = 0;
13   static uint32_t flancos = 0, t_muestreo = 0, t_reporte = 0;
14   uint32_t ahora = millis();
15   if (ahora - t_muestreo >= 2) {
16     t_muestreo = ahora;
17     uint8_t estado = digitalRead(PIN_SONIDO);
18     if (estado != estado_previo) { flancos++; estado_previo = estado; }
19   }
20   if (ahora - t_reporte >= 1000) {
21     t_reporte = ahora;
22     Serial.print(ahora); Serial.print(',');
23     Serial.print(estado_previo); Serial.print(',');
24     Serial.println(flancos);
25     flancos = 0;
26   }
27 }

```

Figura 16. Fragmento de la prueba individual del módulo de sonido.

Fuente: elaboración propia, a partir del código fuente entregado (Zapalloman, 2026).

5.6 Escáner I2C de diagnóstico

Se utilizó un escáner I2C como herramienta de diagnóstico para comprobar la respuesta de los dispositivos durante las pruebas individuales. El BH1750 puede responder en 0x23 o 0x5C y el BMP180 utiliza la dirección 0x77 cuando se encuentra disponible. Esta utilidad permitió comprobar la presencia o ausencia de respuesta de cada módulo sin modificar la adquisición de los demás sensores.

6. Programa integrado

El programa integrado oficial mantiene los cinco sensores contemplados por la evaluación. El BMP180 permanece dentro de la lógica y de la estructura de datos del sistema, pero durante la experiencia no entregó un valor de presión. El programa ya está diseñado para continuar con DHT11, BH1750, MQ135 y sonido cuando esto ocurre; por ello `presion_hPa` permanece como NaN mientras el resto de las variables sigue registrándose normalmente.

```
// Fragmento del programa integrado oficial
Wire.begin();
dht.begin();

bmp_ok = bmpIniciar();
bh_ok = bhIniciar();

if (!bmp_ok)
  Serial.println(F("# ADVERTENCIA: BMP180 no detectado (0x77)"));
if (!bh_ok)
  Serial.println(F("# ADVERTENCIA: BH1750 no detectado (0x23/0x5C)"));

Serial.println(F(
  "tiempo_ms,temperatura_C,humedad_pct,presion_hPa,"
  "lux,mq135_raw,sonido_estado,sonido_flancos_ls"));
```

Figura 17. Inicialización de los sensores en el programa integrado.

Fuente: elaboración propia, a partir del código fuente entregado (Zapalloman, 2026).

```
// El BMP180 permanece contemplado, pero no bloquea la adquisicion
float presion_hPa = NAN;
if (bmp_ok) {
  bmpLeerTemperatura();
  presion_hPa = bmpLeerPresion() / 100.0;
}

// ... se leen los demas sensores normalmente ...
Serial.print(millis());
Serial.print(',');
Serial.print(temperatura, 1);
Serial.print(',');
Serial.print(humedad, 1);
Serial.print(',');
Serial.print(presion_hPa, 2); // NaN si BMP180 no entrega valor
Serial.print(',');
Serial.print(lux, 1);
```

Figura 18. Muestreo conjunto y generación de la fila CSV, incluyendo NaN para presión no disponible.

Fuente: elaboración propia, a partir del código fuente entregado (Zapalloman, 2026).

6.1 Estrategia de integración

- Serial se inicializa a 115200 baudios.
- DHT11 se inicia en D2 y se actualiza cada 2 s.
- El módulo de sonido se supervisa cada 2 ms y sus flancos se resumen cada segundo.
- MQ135 se lee mediante analogRead(A0) en cada ciclo de 1 s.
- BMP180 y BH1750 se manejan como dispositivos I2C mediante conexiones o puertos diferenciados en el montaje realizado; el diagrama integrado definitivo debe reflejar esa separación física.
- Si BMP180 no responde, el programa conserva el resto de la adquisición y escribe NaN en presion_hPa.
- Se transmite una sola fila CSV aproximadamente cada segundo.

6.2 Frecuencia de adquisición

Elemento	Periodicidad / tratamiento
Fila conjunta	≈1 s mediante millis().
DHT11	Lectura cada 2 s; se conserva el último valor válido entre filas.
BMP180	Intento de lectura cada ciclo sólo si bmp_ok=true.
BH1750	Modo continuo; lectura cada fila de 1 s.
MQ135	Una lectura ADC por fila.
Sonido	Muestreo interno cada 2 ms; resumen de estado/flancos cada 1 s.

6.3 Formato serial

Cabecera emitida por el integrado:

tiempo_ms,temperatura_C,humedad_pct,presion_hPa,lux,mq135_raw,sonido_estado,sonido_flancos_1s

El archivo consolidado incorpora adicionalmente la columna segmento para distinguir el bloque A de adquisición integrada y el bloque B correspondiente a una prueba complementaria del MQ135. Esta columna es metadato de organización y no reemplaza ninguna medición del Arduino.

7. Desarrollo experimental y resultados

7.1 Procedimiento experimental

1. Validar los sensores individualmente y revisar las salidas por puerto serial.
2. Cargar el programa integrado y verificar que la cabecera aparezca una sola vez.
3. Registrar la serie principal durante aproximadamente cinco minutos a 1 Hz.
4. Aplicar cambios seguros de iluminación, humedad/ambiente, calidad de aire y eventos acústicos.
5. Conservar una prueba complementaria del MQ135 para observar una variación mayor de su señal.
6. Revisar estructura del CSV, periodicidad y valores inválidos sin completar datos manualmente.

7.2 Resumen cuantitativo del conjunto de datos

Indicador	Resultado
Registros totales	420
Segmento A	300 registros de adquisición integrada
Segmento B	120 registros complementarios MQ135
Periodicidad predominante	≈1000 ms

7.4 Humedad relativa

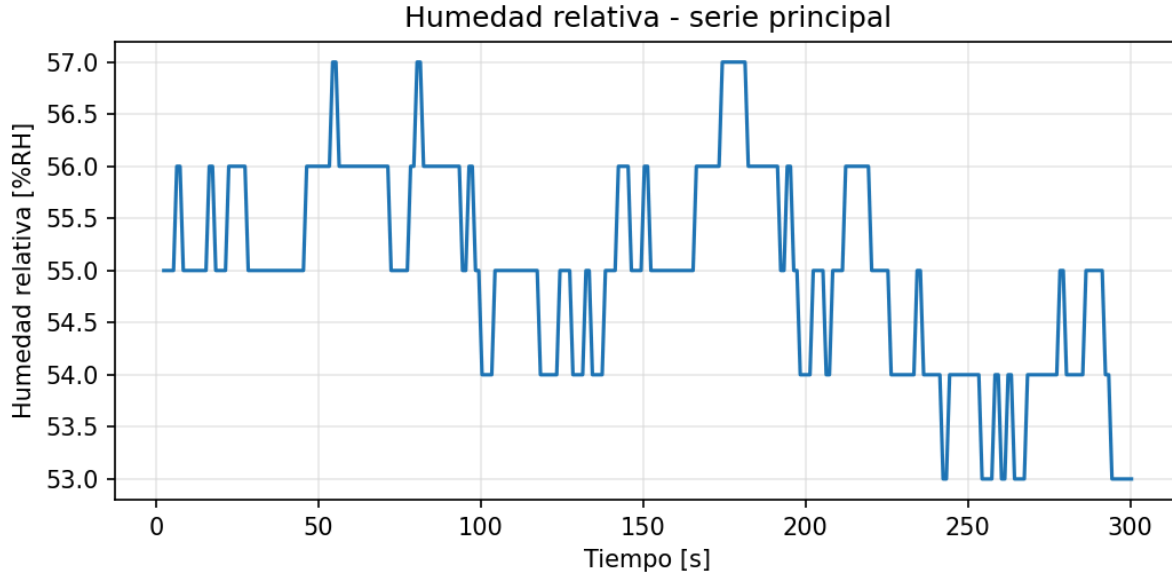


Figura 20. Evolución de humedad relativa durante la serie principal.

Fuente: elaboración propia, a partir de csv_datos_combinados.csv (Equipo de trabajo, 2026).

La humedad relativa se mantuvo entre 53 y 57%RH, con promedio 55.00%RH. La variación observada es coherente con un ambiente interior relativamente estable y cambios moderados durante la experiencia.

7.5 Iluminancia

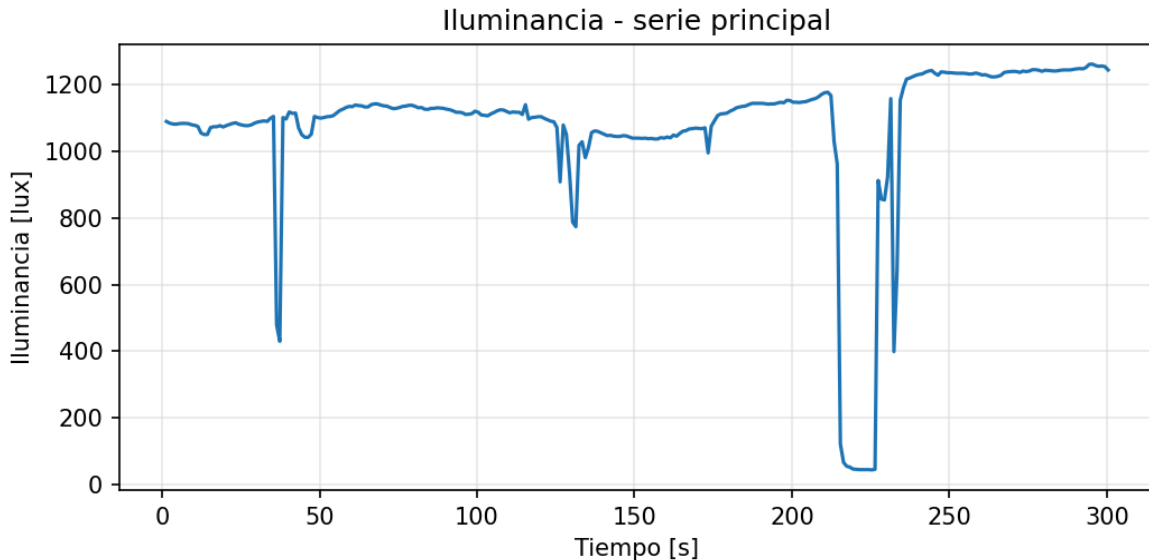


Figura 21. Evolución de iluminancia registrada por el BH1750.

Fuente: elaboración propia, a partir de csv_datos_combinados.csv (Equipo de trabajo, 2026).

La iluminancia varió entre 45 y 1260 lux. El descenso pronunciado visible en la serie evidencia una condición de menor iluminación/sombreado y permite demostrar que el BH1750 respondió al cambio de luz.

7.6 MQ135

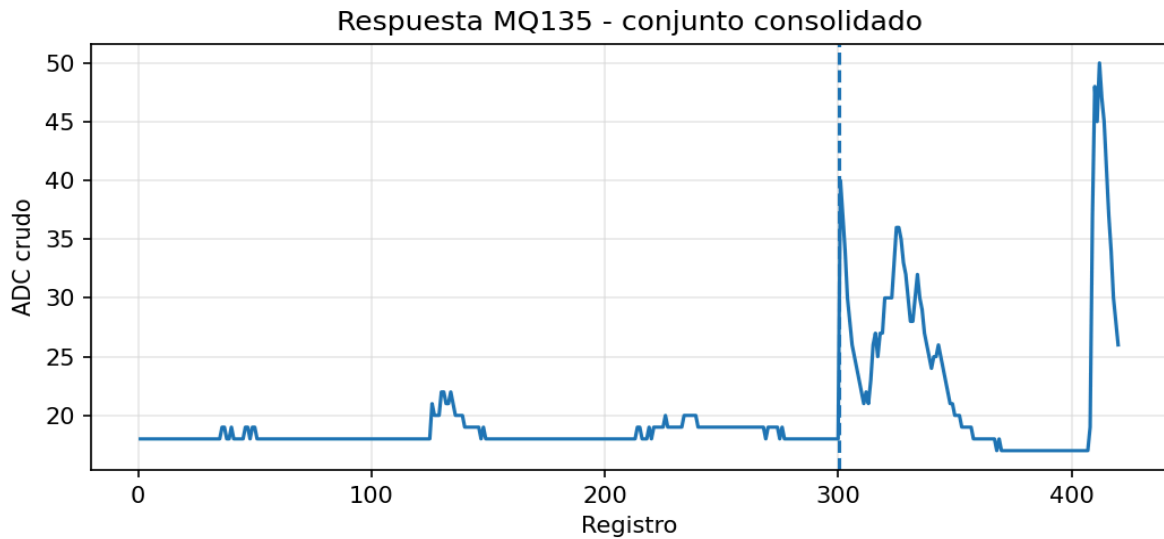


Figura 22. Respuesta ADC del MQ135 en los segmentos A y B; la línea vertical separa ambos bloques.

Fuente: elaboración propia, a partir de csv_datos_combinados.csv (Equipo de trabajo, 2026).

En el segmento A, la lectura se concentró entre 18 y 22 ADC (promedio 18.39). En el segmento B alcanzó 50 ADC y presentó una variación mayor. Estos valores se interpretan únicamente como respuesta ADC cruda, no como concentración en ppm.

7.7 Eventos acústicos

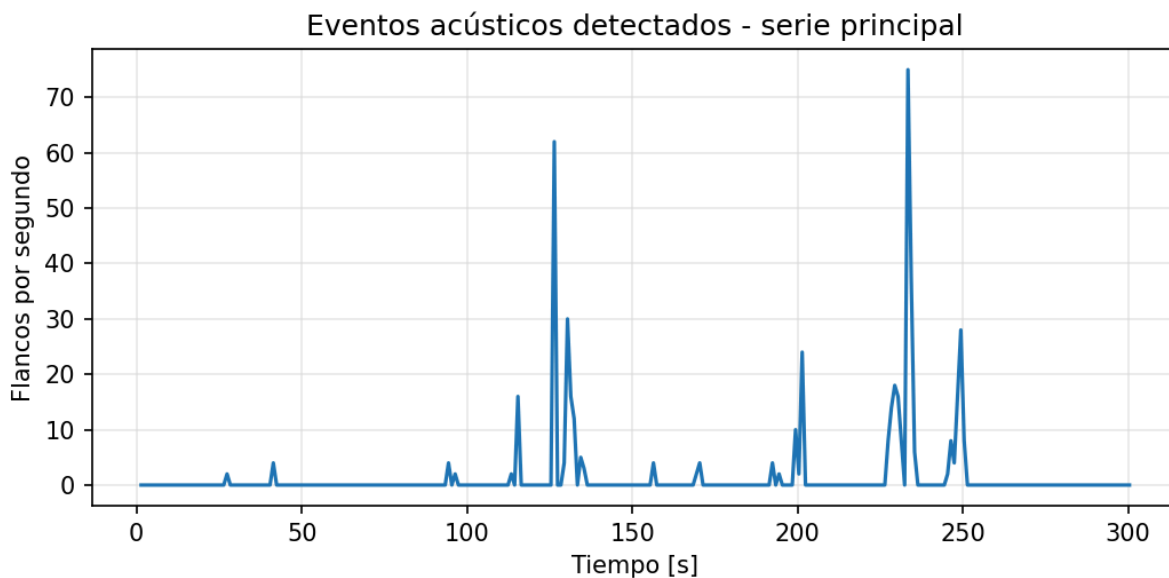


Figura 23. Conteo de cambios de estado del módulo de sonido por segundo.

Fuente: elaboración propia, a partir de csv_datos_combinados.csv (Equipo de trabajo, 2026).

El gráfico muestra los flancos detectados en cada segundo, 35 de las 300 filas registraron al menos un cambio de estado y el máximo fue 75 flancos en un segundo. El conteo evidencia eventos que superaron el umbral del módulo, pero no representa nivel sonoro en dB ni cantidad exacta de aplausos.

7.8 BMP180 y ausencia de presión válida

La columna presion_hPa se mantuvo en la estructura del CSV porque el BMP180 forma parte del diseño y del programa integrado. Sin embargo, el sensor no entregó datos válidos durante la experiencia y la totalidad de sus registros aparece como NaN. El informe conserva estos valores sin reemplazarlos por datos artificiales. La ausencia de presión se considera la principal limitación experimental del montaje.

8. Adquisición y archivo CSV

8.1 Procedimiento de captura

- El Arduino transmite una fila aproximadamente cada 1 s por puerto serial a 115200 baudios.
- Las líneas de advertencia que comienzan con # se excluyen de las filas tabulares.
- El bloque A contiene la adquisición integrada; el bloque B corresponde a una prueba adicional del MQ135.
- El archivo consolidado conserva los valores NaN originales y añade segmento como identificador del bloque experimental.
- Se verifica que todas las filas mantengan el mismo número de campos.

8.2 Definición de columnas

Columna	Origen	Significado	Unidad / dominio
segmento	Consolidación	Identifica bloque A o B	A / B
tiempo_ms	Arduino	Tiempo desde inicio del bloque	ms
temperatura_C	DHT11	Temperatura ambiente	°C
humedad_pct	DHT11	Humedad relativa	%RH
presion_hPa	BMP180	Presión atmosférica; sin datos válidos	hPa / NaN
lux	BH1750	Iluminancia	lux
mq135_raw	MQ135	Lectura ADC cruda	0-1023
sonido_estado	Sonido	Estado digital instantáneo	0/1
sonido_flancos_1s	Procesamiento	Cambios de estado durante 1 s	conteo

8.3 Control de calidad

Control	Resultado	Interpretación
Cantidad total	420 filas	300 integradas + 120 MQ135 complementarias.
Cabecera	Sí	Columnas identificadas.
Periodicidad A	≈1 s	Adecuada para el objetivo de muestreo.
Columnas uniformes	Sí	Todas las filas contienen los mismos campos.
Presión	No válida	420 valores NaN.
Datos sintéticos	No se agregan	Los NaN se conservan como limitación real.

9. Explicación del flujo del código integrado

Etapa	Descripción
Inicialización	Configura Serial, pines, Wire y DHT e inicializa los sensores contemplados por el sistema.
Diagnóstico BMP180	El programa comprueba la disponibilidad del BMP180. Si el sensor no entrega una respuesta utilizable, su estado queda como no disponible y la ejecución del integrado continúa.
Muestreo de sonido	digitalRead(D3) cada 2 ms; cada transición incrementa sonido_flancos.
Actualización DHT11	Cada 2 s se leen temperatura y humedad. Sólo se reemplazan los valores conservados cuando ambos son válidos.
Ciclo de 1 Hz	Cada segundo se lee MQ135, se intenta presión sólo si bmp_ok y se lee BH1750 sólo si bh_ok.
Tratamiento de presión	presion_hPa comienza como NaN. Sólo cambia si el BMP180 entrega una medición utilizable.
Fila CSV	Se imprime tiempo, temperatura, humedad, presión/NaN, lux, MQ135, estado de sonido y flancos.

10. Aporte individual de cada integrante

Uriel Navarrete

Sensor DHT11: investigación de funcionamiento, conexión física y código individual con la biblioteca DHT; validación de temperatura y humedad por puerto serial.

Javier Farías

Integración: desarrollo y coordinación del programa integrado, organización de la fila CSV a 1 Hz, adquisición de datos y consolidación del informe.

Javiera Morán

Sensor BH1750/GY-30: investigación de medición de iluminancia por I2C, conexión y prueba bajo condiciones de iluminación diferentes.

Guillermo Navarro

Sensor MQ135: investigación del sensor de gases, conexión analógica y programación de la lectura ADC cruda, evitando atribuir ppm sin calibración.

Vicente Abarca

Sensor de sonido digital: investigación del funcionamiento por umbral, conexión y programación de detección de cambios de estado.

Gianella Catalán

Sensor BMP180: investigación del funcionamiento barométrico I2C, conexión SDA/SCL/VCC/GND y desarrollo/revisión del código individual para presión.

11. Conclusiones individuales

Uriel Navarrete

Aprendí a integrar un sensor digital que entrega temperatura y humedad mediante un protocolo de una línea. La principal limitación observada es la baja velocidad de actualización del DHT11 respecto del ciclo global. Como mejora, utilizaría un sensor de mayor resolución y una estrategia de adquisición que aproveche mejor la periodicidad del sistema integrado.

Javier Farías

Aprendí los desafíos de integrar interfaces I2C, digital y analógica en una fila de datos a 1 Hz y de mantener el sistema operativo frente a la falla de un sensor. La principal limitación fue la ausencia de presión válida. Como mejora, incorporaría una etapa de autodiagnóstico previa y un procedimiento de sustitución/verificación del hardware antes de comenzar la adquisición.

Javiera Morán

Aprendí a utilizar un sensor digital de iluminancia y a interpretar una variable directamente en lux. La experiencia mostró una variación amplia de iluminación. Como mejora, realizaría pruebas controladas con niveles de luz previamente definidos y documentados.

Guillermo Navarro

Aprendí a distinguir una lectura ADC de una concentración física calibrada. La principal limitación del MQ135 es que su respuesta depende de estabilización y calibración. Como mejora, documentaría una línea base más prolongada y usaría un procedimiento de calibración si el objetivo fuese obtener una magnitud física.

Vicente Abarca

Aprendí a trabajar con una salida digital de umbral y a procesar eventos acústicos mediante cambios de estado. La limitación es que esta señal no cuantifica el nivel sonoro. Como mejora, incorporaría una salida analógica o un sensor de audio apropiado con una metodología de calibración.

Gianella Catalán

Aprendí a trabajar con un sensor barométrico mediante I2C y a relacionar registros y coeficientes de calibración con la presión calculada. La principal limitación fue que el BMP180 no entregó datos válidos, por lo que `presion_hPa` quedó como NaN. Como mejora, validaría el módulo con una unidad funcional antes de iniciar la adquisición principal.

12. Bibliografía

ASAIR (Guangzhou Aosong Electronics Co., Ltd.). (2017). DHT11 Product Manual, Version V1.3 (2017-03-31). [https://www.aosong.com/userfiles/files/media/DHT11-V1_3说明书\(详细版\).pdf](https://www.aosong.com/userfiles/files/media/DHT11-V1_3说明书(详细版).pdf)

Adafruit Industries. (s. f.). DHT sensor library [Repositorio de software]. GitHub. <https://github.com/adafruit/DHT-sensor-library>

Bosch Sensortec GmbH. (2013). BMP180 Digital Pressure Sensor (BST-BMP180-DS000-09, Revision 2.5). <https://cdn-shop.adafruit.com/datasheets/BST-BMP180-DS000-09.pdf>

ROHM Co., Ltd. (2011). BH1750FVI: Digital 16-bit Serial Output Type Ambient Light Sensor IC (Technical Note, Rev. D).

Zhengzhou Winsen Electronics Technology Co., Ltd. (2021). MQ135 Air Quality Gas Sensor Manual (Version 1.6). <https://www.winsen-sensor.com/d/files/manual/mq135.pdf>

Texas Instruments. (2025). LM393B, LM2903B, LM193, LM293, LM393 and LM2903 Dual Comparators (Rev. AH). <https://www.ti.com/lit/ds/symlink/lm393.pdf>

Arduino. (s. f.). Arduino UNO Rev3: Technical specifications.
<https://store.arduino.cc/products/arduino-uno-rev3>

Arduino. (s. f.). Wire - I2C communication library.
<https://docs.arduino.cc/language-reference/en/functions/communication/wire/>

Campusano, E. (2026). Solemne #1: Programación de Microprocesadores (CINF1P09) [Pauta de evaluación]. Universidad Andrés Bello.

Zapalloman. (2026). solemne-1-microprocesadores [Repositorio de código]. GitHub.
<https://github.com/Zapalloman/solemne-1-microprocesadores>

Equipo de trabajo. (2026). csv_datos_combinados.csv [Conjunto de datos experimental no publicado].

13. Anexos

Anexo A. Código fuente

Las páginas siguientes reproducen, mediante capturas completas y consecutivas, el contenido de los archivos de código entregados. Cada conjunto de capturas conserva la numeración de líneas para permitir su trazabilidad. Las capturas corresponden al código fuente de entrega; la disposición física definitiva del montaje debe interpretarse junto con los diagramas de conexión del informe y no a partir de los comentarios del código por sí solos.

El programa integrado utilizado ya contempla la ausencia de una lectura válida del BMP180 sin detener la adquisición de los demás sensores.

A.1 Código individual DHT11 — 01_dht11.ino

01_dht11.ino — captura 1/2

```

1  /*
2  * Solemne 1 - Grupo 1 - DHT11
3  * Sensor: DHT11 (temperatura y humedad relativa)
4  * Conexion: DATA -> D2, VCC -> 5V, GND -> GND
5  *
6  * Requiere la libreria "DHT sensor library" (Adafruit) + "Adafruit Unified Sensor".
7  *
8  * Que muestra por serial:
9  *   - tiempo_ms: milisegundos desde que arranco el programa (lectura cruda de millis())
10 *   - temperatura_C: variable fisica calculada por la libreria (grados Celsius)
11 *   - humedad_pct: variable fisica calculada por la libreria (porcentaje de humedad relativa)
12 *
13 * Nota: el DHT11 entrega internamente un dato DIGITAL de 40 bits
14 * (8 humedad entera + 8 humedad decimal + 8 temperatura entera + 8 decimal + 8 checksum).
15 * No hay "voltaje" que medir: la libreria decodifica ese protocolo de un solo cable.
16 */
17
18 #include <DHT.h>
19
20 #define PIN_DHT 2 // pin de datos del DHT11
21 #define TIPO_DHT DHT11 // modelo del sensor
22
23 DHT dht(PIN_DHT, TIPO_DHT);
24
25 void setup() {
26   Serial.begin(115200);
27   dht.begin();
28
29   // Fila de encabezado (solo una vez, al inicio)
30   Serial.println(F("tiempo_ms,temperatura_C,humedad_pct"));
31 }
32
33 void loop() {
34   // El DHT11 no acepta lecturas mas rapidas que 1 vez por segundo (datasheet Aosong).
35   // Usamos 2 s para dar margen y evitar lecturas invalidas.
36   static uint32_t t_ultima = 0;
37   if (millis() - t_ultima < 2000) return;
38   t_ultima = millis();
39
40   float humedad = dht.readHumidity(); // %RH
41   float temperatura = dht.readTemperature(); // °C (sin argumento = Celsius)
42
43   // Si la lectura falla, la libreria devuelve NaN.

```

Captura A.1 Código individual DHT11-1. Código completo de 01_dht11.ino (parte 1 de 2).

Fuente: elaboración propia, a partir del código fuente entregado (Zapalloman, 2026).

01_dht11.ino - captura 2/2

```
44 // NO se imprime fila de datos: se informa el error como comentario.
45 if (isnan(humedad) || isnan(temperatura)) {
46     Serial.println(F("# DHT11: lectura invalida. Revisar DATA en D2, alimentacion y GND comun.));
47     return;
48 }
49
50 Serial.print(millis());
51 Serial.print(',');
52 Serial.print(temperatura, 1);
53 Serial.print(',');
54 Serial.println(humedad, 1);
55 }
```

Captura A.1 Código individual DHT11-2. Código completo de 01_dht11.ino (parte 2 de 2).

Fuente: elaboración propia, a partir del código fuente entregado (Zapalloman, 2026).

A.2 Código individual BMP180 — 02_bmp180.ino

02_bmp180.ino - captura 1/4

```

1  /*
2  * Solemne 1 - Grupo 1 - BMP180 (presion atmosferica, bus I2C)
3  * Conexion: VCC -> 5V (modulo GY-68 trae regulador y adaptacion de nivel),
4  *           GND -> GND, SDA -> A4, SCL -> A5
5  *
6  * NO requiere libreria externa: se implementa la lectura directa por I2C con Wire,
7  * siguiendo el algoritmo de compensacion del datasheet Bosch BMP180.
8  *
9  * Registros usados:
10 *   0xAA..0xBF : 11 coeficientes de calibracion (22 bytes), propios de cada chip
11 *   0xD0       : chip ID (BMP180 = 0x55)
12 *   0xF4       : registro de control (inicia conversion)
13 *   0xF6..0xF8 : resultado de la conversion
14 *
15 * Salida:
16 *   - presion_hPa : variable fisica (hectopascales)
17 *   - bmp_temp_C  : temperatura interna del chip (se usa para compensar la presion)
18 */
19
20 #include <Wire.h>
21
22 #define DIR_BMP180 0x77 // direccion I2C fija del BMP180
23
24 // Coeficientes de calibracion (se leen del chip al inicio)
25 int32_t ac1, ac2, ac3, b1, b2, mb, mc, md;
26 uint32_t ac4, ac5, ac6;
27
28 int32_t b5; // valor intermedio de temperatura, necesario para compensar presion
29
30 // ----- Utilidades I2C -----
31 uint16_t leer16(uint8_t registro) {
32   Wire.beginTransmission(DIR_BMP180);
33   Wire.write(registro);
34   Wire.endTransmission();
35   Wire.requestFrom((uint8_t)DIR_BMP180, (uint8_t)2);
36   uint16_t valor = ((uint16_t)Wire.read() << 8) | Wire.read();
37   return valor;
38 }
39
40 // Lee temperatura cruda UT y calcula la temperatura compensada (0.1 °C)
41 int32_t leerTemperatura() {
42   Wire.beginTransmission(DIR_BMP180);
43   Wire.write(0xF4);

```

Captura A.2 Código individual BMP180-1. Código completo de 02_bmp180.ino (parte 1 de 4).

Fuente: elaboración propia, a partir del código fuente entregado (Zapalloman, 2026).

02_bmp180.ino - captura 2/4

```

44   Wire.write(0x2E);           // orden: medir temperatura
45   Wire.endTransmission();
46   delay(5);                   // el datasheet indica 4.5 ms maximos
47
48   int32_t ut = (int32_t)leer16(0xF6);
49
50   int32_t x1 = ((ut - (int32_t)ac6) * (int32_t)ac5) >> 15;
51   int32_t x2 = ((int32_t)mc * 2048) / (x1 + md);
52   b5 = x1 + x2;
53   return (b5 + 8) >> 4;       // unidad: 0.1 °C
54 }
55
56 // Lee presion cruda UP (sobremuestreo oss=3, el mas preciso) y la compensa -> Pa
57 int32_t leerPresion() {
58     const uint8_t oss = 3;
59
60     Wire.beginTransmission(DIR_BMP180);
61     Wire.write(0xF4);
62     Wire.write(0x34 | (oss << 6)); // orden: medir presion, oversampling 3
63     Wire.endTransmission();
64     delay(26);                     // oss=3 -> 25.5 ms maximos
65
66     Wire.beginTransmission(DIR_BMP180);
67     Wire.write(0xF6);
68     Wire.endTransmission();
69     Wire.requestFrom((uint8_t)DIR_BMP180, (uint8_t)3);
70     int32_t up = ((int32_t)Wire.read() << 16) | ((int32_t)Wire.read() << 8) | Wire.read();
71     up = up >> (8 - oss);
72
73     // ---- Algoritmo de compensacion del datasheet (aritmetica entera) ----
74     int32_t x1, x2, x3, b3, b6;
75     uint32_t b4, b7;
76
77     b6 = b5 - 4000;
78     x1 = ((int32_t)b2 * (b6 * b6 >> 12)) >> 11;
79     x2 = ((int32_t)ac2 * b6) >> 11;
80     x3 = x1 + x2;
81     b3 = (((int32_t)ac1 * 4 + x3) << oss) + 2) >> 2;
82     x1 = ((int32_t)ac3 * b6) >> 13;
83     x2 = ((int32_t)b1 * ((b6 * b6) >> 12)) >> 16;
84     x3 = ((x1 + x2) + 2) >> 2;
85     b4 = (ac4 * (uint32_t)(x3 + 32768)) >> 15;
86     b7 = ((uint32_t)up - b3) * (50000 >> oss);

```

Captura A.2 Código individual BMP180-2. Código completo de 02_bmp180.ino (parte 2 de 4).

Fuente: elaboración propia, a partir del código fuente entregado (Zapalloman, 2026).

02_bmp180.ino - captura 3/4

```

87
88   int32_t p;
89   if (b7 < 0x80000000UL) {
90       p = (b7 * 2) / b4;
91   } else {
92       p = (b7 / b4) * 2;
93   }
94   x1 = (p >> 8) * (p >> 8);
95   x1 = (x1 * 3038) >> 16;
96   x2 = (-7357 * p) >> 16;
97   p = p + ((x1 + x2 + 3791) >> 4);
98
99   return p;    // unidad: Pascales
100 }
101
102 void setup() {
103     Serial.begin(115200);
104     Wire.begin();
105
106     // Verificar que el chip responde
107     Wire.beginTransmission(DIR_BMP180);
108     if (Wire.endTransmission() != 0) {
109         Serial.println(F("# BMP180 no detectado. Revisar SDA=A4, SCL=A5, alimentacion y GND."));
110         while (true) { delay(1000); }
111     }
112
113     // Verificar chip ID (0x55)
114     uint16_t id = leer16(0xD0);
115     if (id != 0x55) {
116         Serial.print(F("# BMP180: chip ID inesperado 0x"));
117         Serial.println(id, HEX);
118         while (true) { delay(1000); }
119     }
120
121     // Leer coeficientes de calibracion
122     ac1 = (int16_t)leer16(0xAA);
123     ac2 = (int16_t)leer16(0xAC);
124     ac3 = (int16_t)leer16(0xAE);
125     ac4 = leer16(0xB0);
126     ac5 = leer16(0xB2);
127     ac6 = leer16(0xB4);
128     b1 = (int16_t)leer16(0xB6);
129     b2 = (int16_t)leer16(0xB8);

```

Captura A.2 Código individual BMP180-3. Código completo de 02_bmp180.ino (parte 3 de 4).

Fuente: elaboración propia, a partir del código fuente entregado (Zapalloman, 2026).

02_bmp180.ino - captura 4/4

```

130  mb = (int16_t)leer16(0xBA);
131  mc = (int16_t)leer16(0xBC);
132  md = (int16_t)leer16(0xBE);
133  int32_t en1 = (int16_t)leer16(0xBF); // (no usado, solo referencia)
134
135  Serial.println(F("tiempo_ms,presion_hPa,bmp_temp_C"));
136  }
137
138  void loop() {
139    static uint32_t t_ultima = 0;
140    if (millis() - t_ultima < 1000) return;
141    t_ultima = millis();
142
143    int32_t temperatura01 = leerTemperatura(); // 0.1 °C
144    int32_t presion_pa = leerPresion(); // Pa
145
146    Serial.print(millis());
147    Serial.print(',');
148    Serial.print(presion_pa / 100.0, 2); // Pa -> hPa
149    Serial.print(',');
150    Serial.println(temperatura01 / 10.0, 1); // 0.1 °C -> °C
151  }

```

Captura A.2 Código individual BMP180-4. Código completo de 02_bmp180.ino (parte 4 de 4).

Fuente: elaboración propia, a partir del código fuente entregado (Zapalloman, 2026).

A.3 Código individual BH1750 — 03_bh1750.ino

03_bh1750.ino - captura 1/2

```

1  /*
2  * Solemne 1 - Grupo 1 - BH1750 / GY-302 (iluminancia en lux, bus I2C)
3  * Conexion: VCC -> 5V (modulo GY-302), GND -> GND, SDA -> A4, SCL -> A5
4  *
5  * NO requiere libreria externa: lectura directa por I2C con Wire.
6  *
7  * Direcciones posibles: 0x23 (ADDR al aire, tipico) o 0x5C (ADDR a VCC).
8  * El programa detecta automaticamente cual responde.
9  *
10 * Comandos usados:
11 *   0x01 : encender (power on)
12 *   0x10 : modo continuo, resolucion alta (1 lx, 120 ms por medicion)
13 * La hoja de datos indica: lux = valor_crudo / 1.2
14 *
15 * Salida:
16 *   - lux : variable fisica (lectura cruda / 1.2)
17 */
18
19 #include <Wire.h>
20
21 uint8_t dirBH = 0x23;
22
23 bool escribirBH(uint8_t comando) {
24   Wire.beginTransmission(dirBH);
25   Wire.write(comando);
26   return Wire.endTransmission() == 0;   // 0 = el dispositivo respondio (ACK)
27 }
28
29 void setup() {
30   Serial.begin(115200);
31   Wire.begin();
32
33   // Detectar direccion: probar 0x23 y luego 0x5C
34   bool encontrado = false;
35   uint8_t candidatas[2] = {0x23, 0x5C};
36   for (uint8_t i = 0; i < 2; i++) {
37     dirBH = candidatas[i];
38     if (escribirBH(0x01)) {   // power on
39       encontrado = true;
40       break;
41     }
42   }
43 }

```

Captura A.3 Código individual BH1750-1. Código completo de 03_bh1750.ino (parte 1 de 2).

Fuente: elaboración propia, a partir del código fuente entregado (Zapalloman, 2026).

03_bh1750.ino - captura 2/2

```

44   if (!encontrado) {
45       Serial.println(F("# BH1750 no detectado en 0x23 ni 0x5C. Revisar SDA=A4, SCL=A5, VCC y GND."));
46       while (true) { delay(1000); }
47   }
48
49   escribirBH(0x10); // modo continuo, alta resolucion
50   delay(180);       // primera medicion lista (max 180 ms)
51
52   Serial.print(F("# BH1750 detectado en 0x"));
53   Serial.println(dirBH, HEX);
54   Serial.println(F("tiempo_ms,lux"));
55   }
56
57   void loop() {
58       static uint32_t t_ultima = 0;
59       if (millis() - t_ultima < 1000) return;
60       t_ultima = millis();
61
62       Wire.requestFrom(dirBH, (uint8_t)2);
63       if (Wire.available() < 2) {
64           Serial.println(F("# BH1750: sin respuesta al leer lux"));
65           return;
66       }
67       uint16_t crudo = ((uint16_t)Wire.read() << 8) | Wire.read();
68       float lux = crudo / 1.2; // conversion indicada por el fabricante (ROHM)
69
70       Serial.print(millis());
71       Serial.print(',');
72       Serial.println(lux, 1);
73   }

```

Captura A.3 Código individual BH1750-2. Código completo de 03_bh1750.ino (parte 2 de 2).

Fuente: elaboración propia, a partir del código fuente entregado (Zapalloman, 2026).

A.4 Código individual MQ135 — 04_mq135.ino

04_mq135.ino - captura 1/1

```

1  /*
2  * Solemne 1 - Grupo 1 - MQ135
3  * Sensor: MQ135 (respuesta a gases/calidad de aire)
4  * Conexion: A0 -> A0, VCC -> 5V, GND -> GND
5  *
6  * IMPORTANTE (la solemne lo exige): la lectura de analogRead() es una LECTURA ADC CRUDA.
7  * NO se convierte a ppm. Solo se reporta:
8  *   - mq135_raw : valor ADC de 0 a 1023 (resolucion 10 bits del ATmega328P)
9  *   - mq135_volt: voltaje calculado = raw * 5.0 / 1023
10 *
11 * El MQ135 necesita precalentamiento (burn-in). El fabricante recomienda 24-48 h
12 * para estabilizar; en clases bastan unos minutos, pero la lectura deriva al inicio.
13 */
14
15 #define PIN_MQ135 A0
16
17 void setup() {
18   Serial.begin(115200);
19   pinMode(PIN_MQ135, INPUT);
20   Serial.println(F("tiempo_ms,mq135_raw,mq135_volt"));
21 }
22
23 void loop() {
24   static uint32_t t_ultima = 0;
25   if (millis() - t_ultima < 1000) return; // una muestra por segundo
26   t_ultima = millis();
27
28   int raw = analogRead(PIN_MQ135);          // lectura cruda 0..1023 (entero)
29   float voltaje = raw * (5.0 / 1023.0);      // equivalencia voltaje del ADC de 10 bits
30
31   Serial.print(millis());
32   Serial.print(',');
33   Serial.print(raw);
34   Serial.print(',');
35   Serial.println(voltaje, 3);
36 }

```

Captura A.4 Código individual MQ135-1. Código completo de 04_mq135.ino (parte 1 de 1).

Fuente: elaboración propia, a partir del código fuente entregado (Zapalloman, 2026).

A.5 Código individual sensor de sonido — 05_sonido.ino

05_sonido.ino - captura 1/2

```

1  /*
2  * Solemne 1 - Grupo 1 - Sensor de sonido digital
3  * Sensor: modulo de sonido con microfono electret + comparador LM393 (tipo KY-038)
4  * Conexion: D0 -> D3, VCC -> 5V, GND -> GND
5  *
6  * El modulo tiene potenciometro: gira hasta que el LED del modulo apague en silencio
7  * y encienda al aplaudir/hablar fuerte. Ese umbral define la salida digital.
8  *
9  * Salida:
10 *   - estado_sonido: 0/1 instantaneo (la mayoría de los modulos: 1 en reposo, 0 al detectar)
11 *   - flancos: cantidad de cambios de estado detectados en el ultimo segundo
12 *
13 * Se muestrea cada 2 ms para no perder eventos cortos (un aplauso dura decenas de ms).
14 */
15
16 #define PIN_SONIDO 3
17
18 void setup() {
19   Serial.begin(115200);
20   pinMode(PIN_SONIDO, INPUT);
21   Serial.println(F("tiempo_ms,estado_sonido,flancos_ultimo_segundo"));
22 }
23
24 void loop() {
25   static uint8_t estado_previo = 0;
26   static uint32_t flancos = 0;
27   static uint32_t t_ultimo_muestreo = 0;
28   static uint32_t t_ultimo_reporte = 0;
29
30   uint32_t ahora = millis();
31
32   // Muestreo rapido del estado digital (cada 2 ms)
33   if (ahora - t_ultimo_muestreo >= 2) {
34     t_ultimo_muestreo = ahora;
35     uint8_t estado_actual = digitalRead(PIN_SONIDO);
36     if (estado_actual != estado_previo) {
37       flancos++; // hubo un evento acustico que supero el umbral
38       estado_previo = estado_actual;
39     }
40   }
41
42   // Reporte por serial una vez por segundo
43   if (ahora - t_ultimo_reporte >= 1000) {

```

Captura A.5 Código individual sensor de sonido-1. Código completo de 05_sonido.ino (parte 1 de 2).

Fuente: elaboración propia, a partir del código fuente entregado (Zapalloman, 2026).

05_sonido.ino - captura 2/2

```

44     t_ultimo_reporte = ahora;
45     Serial.print(ahora);
46     Serial.print(',');
47     Serial.print(estado_previo);
48     Serial.print(',');
49     Serial.println(flancos);
50     flancos = 0;
51 }
52 }

```

Captura A.5 Código individual sensor de sonido-2. Código completo de 05_sonido.ino (parte 2 de 2).

Fuente: elaboración propia, a partir del código fuente entregado (Zapalloman, 2026).

A.6 Utilidad diagnóstica — 06_escaner_i2c.ino

06_escaner_i2c.ino - captura 1/1

```

1  /*
2  * Solemne 1 - Grupo 1 - Escaner I2C
3  * Utilidad de diagnostico: recorre todas las direcciones I2C y muestra cuales responden.
4  *
5  * Resultado esperado en el montaje del Grupo 1:
6  *   0x23 o 0x5C -> BH1750 (iluminancia)
7  *   0x77       -> BMP180 (presion)
8  *
9  * Conexion para probar el bus: SDA -> A4, SCL -> A5, VCC -> 5V, GND -> GND (de cada modulo).
10 */
11
12 #include <Wire.h>
13
14 void setup() {
15     Serial.begin(115200);
16     Wire.begin();
17     Serial.println(F("Escaner I2C iniciado..."));
18 }
19
20 void loop() {
21     uint8_t encontrados = 0;
22
23     for (uint8_t direccion = 1; direccion < 127; direccion++) {
24         Wire.beginTransmission(direccion);
25         if (Wire.endTransmission() == 0) {
26             Serial.print(F("Dispositivo encontrado en 0x"));
27             if (direccion < 16) Serial.print('0');
28             Serial.println(direccion, HEX);
29             encontrados++;
30         }
31     }
32
33     if (encontrados == 0) {
34         Serial.println(F("No se encontro ningun dispositivo I2C. Revisar cableado SDA/SCL y alimentacion."));
35     }
36
37     Serial.println(F("--- fin del escaneo ---"));
38     delay(3000);
39 }

```

Captura A.6 Utilidad diagnóstica-1. Código completo de 06_escaner_i2c.ino (parte 1 de 1).

Fuente: elaboración propia, a partir del código fuente entregado (Zapalloman, 2026).

A.7 Código integrado oficial — 99_integrado_grupo1.ino

99_integrado_grupo1.ino - captura 1/6

```

1  /*
2  * Solemne 1 - Grupo 1 - Estacion ambiental inteligente (CODIGO INTEGRADO)
3  *
4  * Integra los 5 sensores del Grupo 1 sobre un Arduino Uno:
5  *   - DHT11      -> D2   (digital, protocolo de 1 cable)
6  *   - Sonido     -> D3   (digital, umbral por comparador LM393)
7  *   - MQ135      -> A0   (analogico, ADC crudo - NO ppm)
8  *   - BMP180     -> I2C  (A4=SDA, A5=SCL, direccion 0x77)
9  *   - BH1750     -> I2C  (A4=SDA, A5=SCL, direccion 0x23 o 0x5C)
10 *
11 * Genera UNA fila CSV por segundo con la forma:
12 *   tiempo_ms,temperatura_C,humedad_pct,presion_hPa,lux,mq135_raw,sonido_estado,sonido_flancos_ls
13 *
14 * El DHT11 se lee cada 2 s (límite del datasheet) y se mantiene el ultimo
15 * valor valido entre lecturas. El resto de sensores se leen cada 1 s.
16 * sonido_flancos_ls es una columna DERIVADA: cuenta los cambios de estado
17 * del sensor de sonido en el ultimo segundo (asi no se pierde un aplauso corto).
18 */
19
20 #include <Wire.h>
21 #include <DHT.h>
22
23 // ----- Pines -----
24 #define PIN_DHT      2
25 #define PIN_SONIDO   3
26 #define PIN_MQ135    A0
27 #define TIPO_DHT     DHT11
28
29 // ----- BMP180 (Wire directo) -----
30 #define DIR_BMP180   0x77
31
32 int32_t ac1, ac2, ac3, b1, b2, mb, mc, md;
33 uint32_t ac4, ac5, ac6;
34 int32_t b5;
35
36 // ----- BH1750 (Wire directo) -----
37 uint8_t dirBH = 0x23;
38
39 // ----- DHT -----
40 DHT dht(PIN_DHT, TIPO_DHT);
41
42 // ----- Estado de los sensores I2C -----
43 bool bmp_ok = false;

```

Captura A.7 Código integrado oficial-1. Código completo de 99_integrado_grupo1.ino (parte 1 de 6).

Fuente: elaboración propia, a partir del código fuente entregado (Zapalloman, 2026).

99_integrado_grupo1.ino - captura 2/6

```

44  bool bh_ok  = false;
45
46  // ===== BMP180: utilidades =====
47  uint16_t bmpLeer16(uint8_t registro) {
48      Wire.beginTransmission(DIR_BMP180);
49      Wire.write(registro);
50      Wire.endTransmission();
51      Wire.requestFrom((uint8_t)DIR_BMP180, (uint8_t)2);
52      return ((uint16_t)Wire.read() << 8) | Wire.read();
53  }
54
55  bool bmpIniciar() {
56      Wire.beginTransmission(DIR_BMP180);
57      if (Wire.endTransmission() != 0) return false;
58      if (bmpLeer16(0xD0) != 0x55) return false;
59
60      ac1 = (int16_t)bmpLeer16(0xAA);
61      ac2 = (int16_t)bmpLeer16(0xAC);
62      ac3 = (int16_t)bmpLeer16(0xAE);
63      ac4 = bmpLeer16(0xB0);
64      ac5 = bmpLeer16(0xB2);
65      ac6 = bmpLeer16(0xB4);
66      b1  = (int16_t)bmpLeer16(0xB6);
67      b2  = (int16_t)bmpLeer16(0xB8);
68      mb  = (int16_t)bmpLeer16(0xBA);
69      mc  = (int16_t)bmpLeer16(0xBC);
70      md  = (int16_t)bmpLeer16(0xBE);
71      return true;
72  }
73
74  int32_t bmpLeerTemperatura() {
75      Wire.beginTransmission(DIR_BMP180);
76      Wire.write(0xF4);
77      Wire.write(0x2E);
78      Wire.endTransmission();
79      delay(5);
80
81      int32_t ut = (int32_t)bmpLeer16(0xF6);
82      int32_t x1 = ((ut - (int32_t)ac6) * (int32_t)ac5) >> 15;
83      int32_t x2 = ((int32_t)mc * 2048) / (x1 + md);
84      b5 = x1 + x2;
85      return (b5 + 8) >> 4;    // 0.1 °C
86  }

```

Captura A.7 Código integrado oficial-2. Código completo de 99_integrado_grupo1.ino (parte 2 de 6).

Fuente: elaboración propia, a partir del código fuente entregado (Zapalloman, 2026).

99_integrado_grupo1.ino - captura 3/6

```

87
88 int32_t bmpLeerPresion() {
89     const uint8_t oss = 3;
90     Wire.beginTransmission(DIR_BMP180);
91     Wire.write(0xF4);
92     Wire.write(0x34 | (oss << 6));
93     Wire.endTransmission();
94     delay(26);
95
96     Wire.beginTransmission(DIR_BMP180);
97     Wire.write(0xF6);
98     Wire.endTransmission();
99     Wire.requestFrom((uint8_t)DIR_BMP180, (uint8_t)3);
100    int32_t up = ((int32_t)Wire.read() << 16) | ((int32_t)Wire.read() << 8) | Wire.read();
101    up = up >> (8 - oss);
102
103    int32_t x1, x2, x3, b3, b6;
104    uint32_t b4, b7;
105    b6 = b5 - 4000;
106    x1 = (((int32_t)b2 * (b6 * b6 >> 12)) >> 11;
107    x2 = (((int32_t)ac2 * b6) >> 11;
108    x3 = x1 + x2;
109    b3 = (((int32_t)ac1 * 4 + x3) << oss) + 2) >> 2;
110    x1 = (((int32_t)ac3 * b6) >> 13;
111    x2 = (((int32_t)b1 * ((b6 * b6) >> 12)) >> 16;
112    x3 = (x1 + x2) + 2) >> 2;
113    b4 = (ac4 * (uint32_t)(x3 + 32768)) >> 15;
114    b7 = ((uint32_t)up - b3) * (50000 >> oss);
115
116    int32_t p;
117    if (b7 < 0x80000000UL) p = (b7 * 2) / b4;
118    else p = (b7 / b4) * 2;
119    x1 = (p >> 8) * (p >> 8);
120    x1 = (x1 * 3038) >> 16;
121    x2 = (-7357 * p) >> 16;
122    p = p + ((x1 + x2 + 3791) >> 4);
123
124    return p; // Pascales
125 }
126
127 // ===== BH1750: utilidades =====
128 bool bhEscribir(uint8_t comando) {
129     Wire.beginTransmission(dirBH);

```

Captura A.7 Código integrado oficial-3. Código completo de 99_integrado_grupo1.ino (parte 3 de 6).

Fuente: elaboración propia, a partir del código fuente entregado (Zapalloman, 2026).

99_integrado_grupo1.ino - captura 4/6

```

130   Wire.write(comando);
131   return Wire.endTransmission() == 0;
132 }
133
134 bool bhIniciar() {
135   uint8_t candidatas[2] = {0x23, 0x5C};
136   for (uint8_t i = 0; i < 2; i++) {
137     dirBH = candidatas[i];
138     if (bhEscribir(0x01)) {      // power on
139       bhEscribir(0x10);          // modo continuo alta resolucion
140       delay(180);
141       return true;
142     }
143   }
144   return false;
145 }
146
147 // ===== Programa principal =====
148 void setup() {
149   Serial.begin(115200);
150   pinMode(PIN_SONIDO, INPUT);
151   pinMode(PIN_MQ135, INPUT);
152
153   Wire.begin();
154   dht.begin();
155
156   bmp_ok = bmpIniciar();
157   bh_ok = bhIniciar();
158   if (!bmp_ok) Serial.println(F("# ADVERTENCIA: BMP180 no detectado (0x77)"));
159   if (!bh_ok) Serial.println(F("# ADVERTENCIA: BH1750 no detectado (0x23/0x5C)"));
160
161   // Encabezado del CSV (la primera fila del archivo)
162   Serial.println(F("tiempo_ms,temperatura_C,humedad_pct,presion_hPa,lux,mq135_raw,sonido_estado,sonido_flancos_1s"));
163
164   delay(1200); // dar tiempo al primer muestreo del DHT11
165 }
166
167 void loop() {
168   static uint32_t t_muestra = 0; // control del periodo de 1 s
169   static uint32_t t_dht = 0; // el DHT11 se lee cada 2 s
170   static float temperatura = NAN;
171   static float humedad = NAN;
172   static uint32_t t_sonido = 0; // muestreo rapido del sonido (2 ms)

```

Captura A.7 Código integrado oficial-4. Código completo de 99_integrado_grupo1.ino (parte 4 de 6).

Fuente: elaboración propia, a partir del código fuente entregado (Zapalloman, 2026).

99_integrado_grupo1.ino - captura 5/6

```

173 static uint8_t sonido_prev = 0;
174 static uint32_t sonido_flancos = 0; // cambios de estado en el ultimo segundo
175
176 uint32_t ahora = millis();
177
178 // --- Muestreo rapido del sensor de sonido (cada 2 ms) ---
179 if (ahora - t_sonido >= 2) {
180     t_sonido = ahora;
181     uint8_t estado_actual = digitalRead(PIN_SONIDO);
182     if (estado_actual != sonido_prev) {
183         sonido_flancos++;
184         sonido_prev = estado_actual;
185     }
186 }
187
188 // --- DHT11 cada 2 segundos (manteniendo ultimo valor valido) ---
189 if (ahora - t_dht >= 2000) {
190     t_dht = ahora;
191     float t = dht.readTemperature();
192     float h = dht.readHumidity();
193     if (!isnan(t) && !isnan(h)) {
194         temperatura = t;
195         humedad = h;
196     }
197 }
198
199 // --- Una fila CSV por segundo ---
200 if (ahora - t_muestra < 1000) return;
201 t_muestra = ahora;
202
203 int mq135_raw = analogRead(PIN_MQ135);
204 uint8_t sonido_estado = sonido_prev;
205 uint32_t flancos_segundo = sonido_flancos;
206 sonido_flancos = 0; // se reinicia el contador para el siguiente segundo
207
208 float presion_hPa = NAN;
209 if (bmp_ok) {
210     bmpLeerTemperatura(); // actualiza b5 (necesario para compensar)
211     presion_hPa = bmpLeerPresion() / 100.0;
212 }
213
214 float lux = NAN;
215 if (bh_ok) {

```

Captura A.7 Código integrado oficial-5. Código completo de 99_integrado_grupo1.ino (parte 5 de 6).

Fuente: elaboración propia, a partir del código fuente entregado (Zapalloman, 2026).

99_integrado_grupo1.ino - captura 6/6

```

216     Wire.requestFrom(dirBH, (uint8_t)2);
217     if (Wire.available() >= 2) {
218         uint16_t crudo = ((uint16_t)Wire.read() << 8) | Wire.read();
219         lux = crudo / 1.2;
220     }
221 }
222
223 Serial.print(millis());
224 Serial.print(',');
225 Serial.print(temperatura, 1);
226 Serial.print(',');
227 Serial.print(humedad, 1);
228 Serial.print(',');
229 Serial.print(presion_hPa, 2);
230 Serial.print(',');
231 Serial.print(lux, 1);
232 Serial.print(',');
233 Serial.print(mql35_raw);
234 Serial.print(',');
235 Serial.print(sonido_estado);
236 Serial.print(',');
237 Serial.println(flancos_segundo);
238 }

```

Captura A.7 Código integrado oficial-6. Código completo de 99_integrado_grupo1.ino (parte 6 de 6).

Fuente: elaboración propia, a partir del código fuente entregado (Zapalloman, 2026).

Anexo B. Visualización completa del CSV

Las páginas siguientes reproducen visualmente la totalidad de las 420 filas de csv_datos_combinados.csv. Se mantiene la columna presion_hPa con NaN porque no se obtuvieron lecturas válidas del BMP180. El bloque A corresponde a la adquisición integrada; el bloque B corresponde a la prueba complementaria del MQ135.

Anexo C - Visualización completa del CSV

csv_datos_combinados.csv - filas 1 a 48 de 420

segmento	tiempo_ms	temperatura_C	humedad_pct	presion_hPa	lux	mql35_raw	sonido_estado	sonido_flancos_ls
A	1386	NaN	NaN	NaN	1088.30	18	0	0
A	2386	21.00	55.00	NaN	1083.30	18	0	0
A	3386	21.00	55.00	NaN	1080.80	18	0	0
A	4386	21.00	55.00	NaN	1080.00	18	0	0
A	5386	21.00	55.00	NaN	1081.70	18	0	0
A	6386	22.00	56.00	NaN	1082.50	18	0	0
A	7386	22.00	56.00	NaN	1082.50	18	0	0
A	8386	22.00	55.00	NaN	1081.70	18	0	0
A	9387	22.00	55.00	NaN	1078.30	18	0	0
A	10387	21.00	55.00	NaN	1076.70	18	0	0
A	11387	21.00	55.00	NaN	1073.30	18	0	0
A	12387	21.00	55.00	NaN	1053.30	18	0	0
A	13387	21.00	55.00	NaN	1049.20	18	0	0
A	14387	21.00	55.00	NaN	1049.20	18	0	0
A	15387	21.00	55.00	NaN	1070.00	18	0	0
A	16387	21.00	56.00	NaN	1072.50	18	0	0
A	17387	21.00	56.00	NaN	1072.50	18	0	0
A	18387	21.00	55.00	NaN	1075.80	18	0	0
A	19387	21.00	55.00	NaN	1071.70	18	0	0
A	20387	21.00	55.00	NaN	1075.80	18	0	0
A	21387	21.00	55.00	NaN	1079.20	18	0	0
A	22387	21.00	56.00	NaN	1082.50	18	0	0
A	23387	21.00	56.00	NaN	1084.20	18	0	0
A	24387	21.00	56.00	NaN	1080.00	18	0	0
A	25387	21.00	56.00	NaN	1077.50	18	0	0
A	26387	21.00	56.00	NaN	1075.80	18	0	0
A	27387	21.00	56.00	NaN	1075.80	18	0	2
A	28387	21.00	55.00	NaN	1078.30	18	0	0
A	29387	21.00	55.00	NaN	1083.30	18	0	0
A	30387	21.00	55.00	NaN	1086.70	18	0	0
A	31387	21.00	55.00	NaN	1088.30	18	0	0
A	32387	21.00	55.00	NaN	1090.00	18	0	0
A	33387	21.00	55.00	NaN	1088.30	18	0	0
A	34387	21.00	55.00	NaN	1097.50	18	0	0
A	35387	21.00	55.00	NaN	1103.30	18	0	0
A	36387	21.00	55.00	NaN	479.20	19	0	0
A	37387	21.00	55.00	NaN	430.00	19	0	0
A	38387	22.00	55.00	NaN	1100.00	18	0	0
A	39387	22.00	55.00	NaN	1095.80	18	0	0
A	40387	22.00	55.00	NaN	1116.70	19	0	0
A	41387	22.00	55.00	NaN	1112.50	18	0	4
A	42387	21.00	55.00	NaN	1113.30	18	0	0
A	43387	21.00	55.00	NaN	1068.30	18	0	0
A	44387	22.00	55.00	NaN	1048.30	18	0	0
A	45387	22.00	55.00	NaN	1040.80	18	0	0
A	46387	22.00	56.00	NaN	1040.80	19	0	0
A	47387	22.00	56.00	NaN	1050.00	19	0	0
A	48387	22.00	56.00	NaN	1103.30	18	0	0

Anexo C - Visualización completa del CSV

csv_datos_combinados.csv - filas 49 a 96 de 420

segmento	tiempo_ms	temperatura_C	humedad_pct	presion_hPa	lux	mql35_raw	sonido_estado	sonido_flancos_ls
A	49387	22.00	56.00	NaN	1100.00	19	0	0
A	50387	22.00	56.00	NaN	1098.30	19	0	0
A	51387	22.00	56.00	NaN	1100.00	18	0	0
A	52387	22.00	56.00	NaN	1102.50	18	0	0
A	53387	22.00	56.00	NaN	1103.30	18	0	0
A	54387	22.00	57.00	NaN	1105.80	18	0	0
A	55387	22.00	57.00	NaN	1113.30	18	0	0
A	56387	22.00	56.00	NaN	1120.80	18	0	0
A	57387	22.00	56.00	NaN	1125.00	18	0	0
A	58387	22.00	56.00	NaN	1130.00	18	0	0
A	59387	22.00	56.00	NaN	1133.30	18	0	0
A	60387	22.00	56.00	NaN	1132.50	18	0	0
A	61387	22.00	56.00	NaN	1137.50	18	0	0
A	62387	21.00	56.00	NaN	1135.80	18	0	0
A	63387	21.00	56.00	NaN	1135.00	18	0	0
A	64387	21.00	56.00	NaN	1131.70	18	0	0
A	65387	21.00	56.00	NaN	1131.70	18	0	0
A	66387	21.00	56.00	NaN	1138.30	18	0	0
A	67387	21.00	56.00	NaN	1140.80	18	0	0
A	68387	21.00	56.00	NaN	1140.80	18	0	0
A	69387	21.00	56.00	NaN	1137.50	18	0	0
A	70387	21.00	56.00	NaN	1135.00	18	0	0
A	71387	21.00	56.00	NaN	1134.20	18	0	0
A	72387	21.00	55.00	NaN	1130.00	18	0	0
A	73387	21.00	55.00	NaN	1126.70	18	0	0
A	74387	21.00	55.00	NaN	1127.50	18	0	0
A	75387	21.00	55.00	NaN	1130.00	18	0	0
A	76387	21.00	55.00	NaN	1133.30	18	0	0
A	77387	21.00	55.00	NaN	1134.20	18	0	0
A	78387	21.00	56.00	NaN	1136.70	18	0	0
A	79387	21.00	56.00	NaN	1136.70	18	0	0
A	80387	21.00	57.00	NaN	1133.30	18	0	0
A	81387	21.00	57.00	NaN	1129.20	18	0	0
A	82387	21.00	56.00	NaN	1130.00	18	0	0
A	83387	21.00	56.00	NaN	1125.00	18	0	0
A	84387	21.00	56.00	NaN	1124.20	18	0	0
A	85387	21.00	56.00	NaN	1127.50	18	0	0
A	86387	22.00	56.00	NaN	1127.50	18	0	0
A	87387	22.00	56.00	NaN	1129.20	18	0	0
A	88387	22.00	56.00	NaN	1128.30	18	0	0
A	89387	22.00	56.00	NaN	1127.50	18	0	0
A	90387	22.00	56.00	NaN	1125.00	18	0	0
A	91387	22.00	56.00	NaN	1123.30	18	0	0
A	92387	22.00	56.00	NaN	1129.00	18	0	0
A	93387	22.00	56.00	NaN	1115.80	18	0	0
A	94387	22.00	55.00	NaN	1115.80	18	0	4
A	95387	22.00	55.00	NaN	1114.20	18	0	0
A	96387	22.00	56.00	NaN	1109.20	18	0	2

Anexo C - Visualización completa del CSV

csv_datos_combinados.csv - filas 97 a 144 de 420

segmento	tiempo_ms	temperatura_C	humedad_pct	presion_hPa	lux	mql35_raw	sonido_estado	sonido_flancos_ls
A	97387	22.00	56.00	NaN	1110.00	18	0	0
A	98387	22.00	55.00	NaN	1111.70	18	0	0
A	99387	22.00	55.00	NaN	1119.20	18	0	0
A	100387	22.00	54.00	NaN	1115.80	18	0	0
A	101387	22.00	54.00	NaN	1107.50	18	0	0
A	102387	22.00	54.00	NaN	1106.70	18	0	0
A	103387	22.00	54.00	NaN	1105.00	18	0	0
A	104387	22.00	55.00	NaN	1110.80	18	0	0
A	105387	22.00	55.00	NaN	1115.00	18	0	0
A	106387	22.00	55.00	NaN	1120.00	18	0	0
A	107387	22.00	55.00	NaN	1123.30	18	0	0
A	108387	22.00	55.00	NaN	1122.50	18	0	0
A	109387	22.00	55.00	NaN	1118.30	18	0	0
A	110387	22.00	55.00	NaN	1114.20	18	0	0
A	111387	22.00	55.00	NaN	1116.70	18	0	0
A	112387	22.00	55.00	NaN	1115.80	18	0	0
A	113387	22.00	55.00	NaN	1115.80	18	0	2
A	114387	22.00	55.00	NaN	1109.20	18	0	0
A	115387	22.00	55.00	NaN	1138.30	18	0	16
A	116387	22.00	55.00	NaN	1095.00	18	0	0
A	117387	22.00	55.00	NaN	1100.00	18	0	0
A	118387	22.00	54.00	NaN	1100.80	18	0	0
A	119387	22.00	54.00	NaN	1102.50	18	0	0
A	120387	21.00	54.00	NaN	1102.50	18	0	0
A	121387	21.00	54.00	NaN	1097.50	18	0	0
A	122387	22.00	54.00	NaN	1093.30	18	0	0
A	123387	22.00	54.00	NaN	1089.20	18	0	0
A	124387	22.00	55.00	NaN	1087.50	18	0	0
A	125387	22.00	55.00	NaN	1070.00	18	0	0
A	126387	22.00	55.00	NaN	907.50	21	0	62
A	127387	22.00	55.00	NaN	1077.50	20	0	0
A	128387	22.00	54.00	NaN	1049.20	20	0	0
A	129387	22.00	54.00	NaN	933.30	20	0	4
A	130387	22.00	54.00	NaN	786.70	22	0	30
A	131387	22.00	54.00	NaN	773.30	22	0	16
A	132387	22.00	55.00	NaN	1016.70	21	0	12
A	133387	22.00	55.00	NaN	1027.50	21	0	0
A	134387	22.00	54.00	NaN	980.00	22	1	5
A	135387	22.00	54.00	NaN	1010.00	21	0	3
A	136387	22.00	54.00	NaN	1055.00	20	0	0
A	137387	22.00	54.00	NaN	1060.00	20	0	0
A	138387	21.00	55.00	NaN	1058.30	20	0	0
A	139387	21.00	55.00	NaN	1054.20	20	0	0
A	140387	21.00	55.00	NaN	1050.00	19	0	0
A	141387	21.00	55.00	NaN	1045.80	19	0	0
A	142387	21.00	56.00	NaN	1046.70	19	0	0
A	143387	21.00	56.00	NaN	1044.20	19	0	0
A	144387	21.00	56.00	NaN	1043.30	19	0	0

Anexo C - Visualización completa del CSV

csv_datos_combinados.csv - filas 145 a 192 de 420

segmento	tiempo_ms	temperatura_C	humedad_pct	presion_hPa	lux	mql35_raw	sonido_estado	sonido_flancos_ls
A	145387	21.00	56.00	NaN	1043.30	19	0	0
A	146387	21.00	55.00	NaN	1045.80	19	0	0
A	147387	21.00	55.00	NaN	1045.00	18	0	0
A	148387	22.00	55.00	NaN	1041.70	19	0	0
A	149387	22.00	55.00	NaN	1038.30	18	0	0
A	150387	22.00	56.00	NaN	1038.30	18	0	0
A	151387	22.00	56.00	NaN	1038.30	18	0	0
A	152387	22.00	55.00	NaN	1037.50	18	0	0
A	153387	22.00	55.00	NaN	1038.30	18	0	0
A	154387	22.00	55.00	NaN	1036.70	18	0	0
A	155387	22.00	55.00	NaN	1037.50	18	0	0
A	156387	22.00	55.00	NaN	1035.80	18	0	4
A	157387	22.00	55.00	NaN	1035.80	18	0	0
A	158387	22.00	55.00	NaN	1040.00	18	0	0
A	159387	22.00	55.00	NaN	1038.30	18	0	0
A	160387	22.00	55.00	NaN	1041.70	18	0	0
A	161387	22.00	55.00	NaN	1039.20	18	0	0
A	162387	22.00	55.00	NaN	1047.50	18	0	0
A	163387	22.00	55.00	NaN	1044.20	18	0	0
A	164387	22.00	55.00	NaN	1052.50	18	0	0
A	165387	22.00	55.00	NaN	1059.20	18	0	0
A	166387	22.00	56.00	NaN	1060.80	18	0	0
A	167387	22.00	56.00	NaN	1065.80	18	0	0
A	168387	22.00	56.00	NaN	1066.70	18	0	0
A	169387	22.00	56.00	NaN	1068.30	18	0	2
A	170387	22.00	56.00	NaN	1067.50	18	0	4
A	171387	22.00	56.00	NaN	1066.70	18	0	0
A	172387	22.00	56.00	NaN	1069.20	18	0	0
A	173387	22.00	56.00	NaN	994.20	18	0	0
A	174387	22.00	57.00	NaN	1074.20	18	0	0
A	175387	22.00	57.00	NaN	1090.00	18	0	0
A	176387	22.00	57.00	NaN	1105.80	18	0	0
A	177387	22.00	57.00	NaN	1110.80	18	0	0
A	178387	22.00	57.00	NaN	1111.70	18	0	0
A	179387	22.00	57.00	NaN	1113.30	18	0	0
A	180387	22.00	57.00	NaN	1119.20	18	0	0
A	181387	22.00	57.00	NaN	1123.30	18	0	0
A	182387	22.00	56.00	NaN	1129.20	18	0	0
A	183387	22.00	56.00	NaN	1131.70	18	0	0
A	184387	22.00	56.00	NaN	1133.30	18	0	0
A	185387	22.00	56.00	NaN	1135.00	18	0	0
A	186387	22.00	56.00	NaN	1139.20	18	0	0
A	187387	22.00	56.00	NaN	1142.50	18	0	0
A	188387	22.00	56.00	NaN	1142.50	18	0	0
A	189387	22.00	56.00	NaN	1142.50	18	0	0
A	190387	22.00	56.00	NaN	1142.50	18	0	0
A	191387	22.00	56.00	NaN	1141.70	18	0	0
A	192387	22.00	55.00	NaN	1140.00	18	0	4

Anexo C - Visualización completa del CSV

csv_datos_combinados.csv - filas 193 a 240 de 420

segmento	tiempo_ms	temperatura_C	humedad_pct	presion_hPa	lux	mql35_raw	sonido_estado	sonido_flancos_ls
A	193387	22.00	55.00	NaN	1140.80	18	0	0
A	194387	22.00	56.00	NaN	1140.80	18	0	2
A	195387	22.00	56.00	NaN	1143.30	18	0	0
A	196387	22.00	55.00	NaN	1145.80	18	0	0
A	197387	22.00	55.00	NaN	1144.20	18	0	0
A	198387	22.00	54.00	NaN	1151.70	18	0	0
A	199387	22.00	54.00	NaN	1150.80	18	0	10
A	200387	22.00	54.00	NaN	1145.80	18	0	2
A	201387	22.00	54.00	NaN	1145.80	18	0	24
A	202387	23.00	55.00	NaN	1145.00	18	0	0
A	203387	23.00	55.00	NaN	1146.70	18	0	0
A	204387	22.00	55.00	NaN	1147.50	18	0	0
A	205387	22.00	55.00	NaN	1150.80	18	0	0
A	206387	22.00	54.00	NaN	1154.20	18	0	0
A	207387	22.00	54.00	NaN	1156.70	18	0	0
A	208387	22.00	55.00	NaN	1161.70	18	0	0
A	209387	22.00	55.00	NaN	1169.20	18	0	0
A	210387	22.00	55.00	NaN	1174.20	18	0	0
A	211387	22.00	55.00	NaN	1175.80	18	0	0
A	212387	22.00	56.00	NaN	1165.80	18	0	0
A	213387	22.00	56.00	NaN	1028.30	18	0	0
A	214387	22.00	56.00	NaN	960.80	19	0	0
A	215387	22.00	56.00	NaN	122.50	19	0	0
A	216387	22.00	56.00	NaN	67.50	18	0	0
A	217387	22.00	56.00	NaN	55.80	18	0	0
A	218387	22.00	56.00	NaN	53.30	18	0	0
A	219387	22.00	56.00	NaN	47.50	19	0	0
A	220387	22.00	55.00	NaN	46.70	18	0	0
A	221387	22.00	55.00	NaN	45.80	19	0	0
A	222387	22.00	55.00	NaN	45.80	19	0	0
A	223387	22.00	55.00	NaN	45.80	19	0	0
A	224387	22.00	55.00	NaN	45.80	19	0	0
A	225387	22.00	55.00	NaN	45.00	19	0	0
A	226387	22.00	54.00	NaN	46.70	20	0	0
A	227387	22.00	54.00	NaN	911.70	19	0	8
A	228387	23.00	54.00	NaN	855.80	19	0	14
A	229387	23.00	54.00	NaN	853.30	19	0	18
A	230387	23.00	54.00	NaN	922.50	19	0	16
A	231387	23.00	54.00	NaN	1156.70	19	0	8
A	232387	23.00	54.00	NaN	399.20	19	0	0
A	233387	23.00	54.00	NaN	643.30	19	1	75
A	234387	22.00	55.00	NaN	1151.70	20	0	37
A	235387	22.00	55.00	NaN	1188.30	20	0	6
A	236387	22.00	54.00	NaN	1215.00	20	0	0
A	237387	22.00	54.00	NaN	1218.30	20	0	0
A	238387	22.00	54.00	NaN	1222.50	20	0	0
A	239387	22.00	54.00	NaN	1226.70	20	0	0
A	240387	22.00	54.00	NaN	1229.20	19	0	0

Anexo C - Visualización completa del CSV

csv_datos_combinados.csv - filas 241 a 288 de 420

segmento	tiempo_ms	temperatura_C	humedad_pct	presion_hPa	lux	mq135_raw	sonido_estado	sonido_flancos_ls
A	241387	22.00	54.00	NaN	1230.80	19	0	0
A	242387	22.00	53.00	NaN	1235.80	19	0	0
A	243387	22.00	53.00	NaN	1239.20	19	0	0
A	244387	22.00	54.00	NaN	1240.80	19	0	0
A	245387	22.00	54.00	NaN	1232.50	19	0	2
A	246387	22.00	54.00	NaN	1226.70	19	0	8
A	247387	22.00	54.00	NaN	1236.70	19	0	4
A	248387	22.00	54.00	NaN	1235.80	19	0	16
A	249387	22.00	54.00	NaN	1234.20	19	0	28
A	250387	22.00	54.00	NaN	1234.20	19	0	8
A	251387	22.00	54.00	NaN	1233.30	19	0	0
A	252387	22.00	54.00	NaN	1232.50	19	0	0
A	253387	22.00	54.00	NaN	1232.50	19	0	0
A	254387	22.00	53.00	NaN	1232.50	19	0	0
A	255387	22.00	53.00	NaN	1231.70	19	0	0
A	256387	22.00	53.00	NaN	1230.00	19	0	0
A	257387	22.00	53.00	NaN	1230.80	19	0	0
A	258387	22.00	54.00	NaN	1233.30	19	0	0
A	259387	22.00	54.00	NaN	1230.80	19	0	0
A	260387	22.00	53.00	NaN	1227.50	19	0	0
A	261387	22.00	53.00	NaN	1228.30	19	0	0
A	262387	22.00	54.00	NaN	1225.00	19	0	0
A	263387	22.00	54.00	NaN	1221.70	19	0	0
A	264387	23.00	53.00	NaN	1221.70	19	0	0
A	265387	23.00	53.00	NaN	1223.30	19	0	0
A	266387	23.00	53.00	NaN	1226.70	19	0	0
A	267387	23.00	53.00	NaN	1235.00	19	0	0
A	268387	23.00	54.00	NaN	1236.70	19	0	0
A	269387	23.00	54.00	NaN	1237.50	18	0	0
A	270387	23.00	54.00	NaN	1238.30	19	0	0
A	271387	23.00	54.00	NaN	1237.50	19	0	0
A	272387	23.00	54.00	NaN	1235.00	19	0	0
A	273387	23.00	54.00	NaN	1239.20	19	0	0
A	274387	23.00	54.00	NaN	1237.50	19	0	0
A	275387	23.00	54.00	NaN	1239.20	18	0	0
A	276387	22.00	54.00	NaN	1243.30	19	0	0
A	277387	22.00	54.00	NaN	1243.30	18	0	0
A	278387	22.00	55.00	NaN	1241.70	18	0	0
A	279387	22.00	55.00	NaN	1238.30	18	0	0
A	280387	23.00	54.00	NaN	1241.70	18	0	0
A	281387	23.00	54.00	NaN	1240.80	18	0	0
A	282387	23.00	54.00	NaN	1240.00	18	0	0
A	283387	23.00	54.00	NaN	1239.20	18	0	0
A	284387	23.00	54.00	NaN	1240.00	18	0	0
A	285387	23.00	54.00	NaN	1241.70	18	0	0
A	286387	23.00	55.00	NaN	1242.50	18	0	0
A	287387	23.00	55.00	NaN	1242.50	18	0	0
A	288387	23.00	55.00	NaN	1242.50	18	0	0

Anexo C - Visualización completa del CSV

csv_datos_combinados.csv - filas 289 a 336 de 420

segmento	tiempo_ms	temperatura_C	humedad_pct	presion_hPa	lux	mql35_raw	sonido_estado	sonido_flancos_ls
A	289387	23.00	55.00	NaN	1244.20	18	0	0
A	290387	22.00	55.00	NaN	1245.80	18	0	0
A	291387	22.00	55.00	NaN	1246.70	18	0	0
A	292387	22.00	54.00	NaN	1245.80	18	0	0
A	293387	22.00	54.00	NaN	1250.00	18	0	0
A	294387	22.00	53.00	NaN	1259.20	18	0	0
A	295387	22.00	53.00	NaN	1260.00	18	0	0
A	296387	23.00	53.00	NaN	1255.80	18	0	0
A	297387	23.00	53.00	NaN	1253.30	18	0	0
A	298387	22.00	53.00	NaN	1254.20	18	0	0
A	299387	22.00	53.00	NaN	1252.50	18	0	0
A	300387	22.00	53.00	NaN	1242.50	18	0	0
B	1000	NaN	NaN	NaN	NaN	40	NaN	NaN
B	2000	NaN	NaN	NaN	NaN	37	NaN	NaN
B	3000	NaN	NaN	NaN	NaN	34	NaN	NaN
B	4000	NaN	NaN	NaN	NaN	30	NaN	NaN
B	5000	NaN	NaN	NaN	NaN	28	NaN	NaN
B	6000	NaN	NaN	NaN	NaN	26	NaN	NaN
B	7000	NaN	NaN	NaN	NaN	25	NaN	NaN
B	8000	NaN	NaN	NaN	NaN	24	NaN	NaN
B	9000	NaN	NaN	NaN	NaN	23	NaN	NaN
B	10000	NaN	NaN	NaN	NaN	22	NaN	NaN
B	11000	NaN	NaN	NaN	NaN	21	NaN	NaN
B	12000	NaN	NaN	NaN	NaN	22	NaN	NaN
B	13000	NaN	NaN	NaN	NaN	21	NaN	NaN
B	14000	NaN	NaN	NaN	NaN	23	NaN	NaN
B	15000	NaN	NaN	NaN	NaN	26	NaN	NaN
B	16000	NaN	NaN	NaN	NaN	27	NaN	NaN
B	17000	NaN	NaN	NaN	NaN	25	NaN	NaN
B	18000	NaN	NaN	NaN	NaN	27	NaN	NaN
B	19000	NaN	NaN	NaN	NaN	27	NaN	NaN
B	20000	NaN	NaN	NaN	NaN	30	NaN	NaN
B	21000	NaN	NaN	NaN	NaN	30	NaN	NaN
B	22000	NaN	NaN	NaN	NaN	30	NaN	NaN
B	23000	NaN	NaN	NaN	NaN	30	NaN	NaN
B	24000	NaN	NaN	NaN	NaN	33	NaN	NaN
B	25000	NaN	NaN	NaN	NaN	36	NaN	NaN
B	26000	NaN	NaN	NaN	NaN	36	NaN	NaN
B	27000	NaN	NaN	NaN	NaN	35	NaN	NaN
B	28000	NaN	NaN	NaN	NaN	33	NaN	NaN
B	29000	NaN	NaN	NaN	NaN	32	NaN	NaN
B	30000	NaN	NaN	NaN	NaN	30	NaN	NaN
B	31000	NaN	NaN	NaN	NaN	28	NaN	NaN
B	32000	NaN	NaN	NaN	NaN	28	NaN	NaN
B	33000	NaN	NaN	NaN	NaN	30	NaN	NaN
B	34000	NaN	NaN	NaN	NaN	32	NaN	NaN
B	35000	NaN	NaN	NaN	NaN	30	NaN	NaN
B	36000	NaN	NaN	NaN	NaN	29	NaN	NaN

Anexo C - Visualización completa del CSV

csv_datos_combinados.csv - filas 337 a 384 de 420

segmento	tiempo_ms	temperatura_C	humedad_pct	presion_hPa	lux	mq135_raw	sonido_estado	sonido_flancos_1s
B	37000	NaN	NaN	NaN	NaN	27	NaN	NaN
B	38000	NaN	NaN	NaN	NaN	26	NaN	NaN
B	39000	NaN	NaN	NaN	NaN	25	NaN	NaN
B	40000	NaN	NaN	NaN	NaN	24	NaN	NaN
B	41000	NaN	NaN	NaN	NaN	25	NaN	NaN
B	42000	NaN	NaN	NaN	NaN	25	NaN	NaN
B	43000	NaN	NaN	NaN	NaN	26	NaN	NaN
B	44000	NaN	NaN	NaN	NaN	25	NaN	NaN
B	45000	NaN	NaN	NaN	NaN	24	NaN	NaN
B	46000	NaN	NaN	NaN	NaN	23	NaN	NaN
B	47000	NaN	NaN	NaN	NaN	22	NaN	NaN
B	48000	NaN	NaN	NaN	NaN	21	NaN	NaN
B	49000	NaN	NaN	NaN	NaN	21	NaN	NaN
B	50000	NaN	NaN	NaN	NaN	20	NaN	NaN
B	51000	NaN	NaN	NaN	NaN	20	NaN	NaN
B	52000	NaN	NaN	NaN	NaN	20	NaN	NaN
B	53000	NaN	NaN	NaN	NaN	19	NaN	NaN
B	54000	NaN	NaN	NaN	NaN	19	NaN	NaN
B	55000	NaN	NaN	NaN	NaN	19	NaN	NaN
B	56000	NaN	NaN	NaN	NaN	19	NaN	NaN
B	57000	NaN	NaN	NaN	NaN	19	NaN	NaN
B	58000	NaN	NaN	NaN	NaN	18	NaN	NaN
B	59000	NaN	NaN	NaN	NaN	18	NaN	NaN
B	60000	NaN	NaN	NaN	NaN	18	NaN	NaN
B	61000	NaN	NaN	NaN	NaN	18	NaN	NaN
B	62000	NaN	NaN	NaN	NaN	18	NaN	NaN
B	63000	NaN	NaN	NaN	NaN	18	NaN	NaN
B	64000	NaN	NaN	NaN	NaN	18	NaN	NaN
B	65000	NaN	NaN	NaN	NaN	18	NaN	NaN
B	66000	NaN	NaN	NaN	NaN	18	NaN	NaN
B	67000	NaN	NaN	NaN	NaN	18	NaN	NaN
B	68000	NaN	NaN	NaN	NaN	17	NaN	NaN
B	69000	NaN	NaN	NaN	NaN	18	NaN	NaN
B	70000	NaN	NaN	NaN	NaN	17	NaN	NaN
B	71000	NaN	NaN	NaN	NaN	17	NaN	NaN
B	72000	NaN	NaN	NaN	NaN	17	NaN	NaN
B	73000	NaN	NaN	NaN	NaN	17	NaN	NaN
B	74000	NaN	NaN	NaN	NaN	17	NaN	NaN
B	75000	NaN	NaN	NaN	NaN	17	NaN	NaN
B	76000	NaN	NaN	NaN	NaN	17	NaN	NaN
B	77000	NaN	NaN	NaN	NaN	17	NaN	NaN
B	78000	NaN	NaN	NaN	NaN	17	NaN	NaN
B	79000	NaN	NaN	NaN	NaN	17	NaN	NaN
B	80000	NaN	NaN	NaN	NaN	17	NaN	NaN
B	81000	NaN	NaN	NaN	NaN	17	NaN	NaN
B	82000	NaN	NaN	NaN	NaN	17	NaN	NaN
B	83000	NaN	NaN	NaN	NaN	17	NaN	NaN
B	84000	NaN	NaN	NaN	NaN	17	NaN	NaN

Anexo C - Visualización completa del CSV

csv_datos_combinados.csv - filas 385 a 420 de 420

segmento	tiempo_ms	temperatura_C	humedad_pct	presion_hPa	lux	mq135_raw	sonido_estado	sonido_flancos_ls
B	85000	NaN	NaN	NaN	NaN	17	NaN	NaN
B	86000	NaN	NaN	NaN	NaN	17	NaN	NaN
B	87000	NaN	NaN	NaN	NaN	17	NaN	NaN
B	88000	NaN	NaN	NaN	NaN	17	NaN	NaN
B	89000	NaN	NaN	NaN	NaN	17	NaN	NaN
B	90000	NaN	NaN	NaN	NaN	17	NaN	NaN
B	91000	NaN	NaN	NaN	NaN	17	NaN	NaN
B	92000	NaN	NaN	NaN	NaN	17	NaN	NaN
B	93000	NaN	NaN	NaN	NaN	17	NaN	NaN
B	94000	NaN	NaN	NaN	NaN	17	NaN	NaN
B	95000	NaN	NaN	NaN	NaN	17	NaN	NaN
B	96000	NaN	NaN	NaN	NaN	17	NaN	NaN
B	97000	NaN	NaN	NaN	NaN	17	NaN	NaN
B	98000	NaN	NaN	NaN	NaN	17	NaN	NaN
B	99000	NaN	NaN	NaN	NaN	17	NaN	NaN
B	100000	NaN	NaN	NaN	NaN	17	NaN	NaN
B	101000	NaN	NaN	NaN	NaN	17	NaN	NaN
B	102000	NaN	NaN	NaN	NaN	17	NaN	NaN
B	103000	NaN	NaN	NaN	NaN	17	NaN	NaN
B	104000	NaN	NaN	NaN	NaN	17	NaN	NaN
B	105000	NaN	NaN	NaN	NaN	17	NaN	NaN
B	106000	NaN	NaN	NaN	NaN	17	NaN	NaN
B	107000	NaN	NaN	NaN	NaN	17	NaN	NaN
B	108000	NaN	NaN	NaN	NaN	19	NaN	NaN
B	109000	NaN	NaN	NaN	NaN	37	NaN	NaN
B	110000	NaN	NaN	NaN	NaN	48	NaN	NaN
B	111000	NaN	NaN	NaN	NaN	45	NaN	NaN
B	112000	NaN	NaN	NaN	NaN	50	NaN	NaN
B	113000	NaN	NaN	NaN	NaN	47	NaN	NaN
B	114000	NaN	NaN	NaN	NaN	45	NaN	NaN
B	115000	NaN	NaN	NaN	NaN	41	NaN	NaN
B	116000	NaN	NaN	NaN	NaN	37	NaN	NaN
B	117000	NaN	NaN	NaN	NaN	34	NaN	NaN
B	118000	NaN	NaN	NaN	NaN	30	NaN	NaN
B	119000	NaN	NaN	NaN	NaN	28	NaN	NaN
B	120000	NaN	NaN	NaN	NaN	26	NaN	NaN